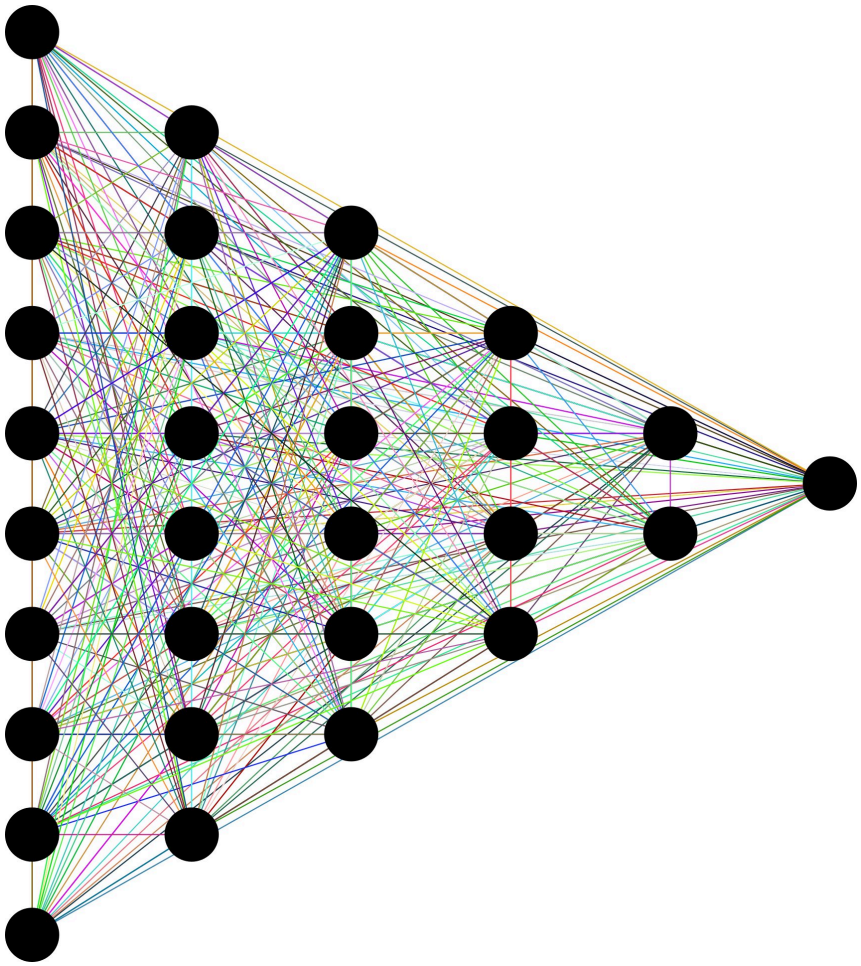
Università degli Studi di Verona

# Intelligenza Artificiale

Appunti del corso 2026/2027



3.14

Anno Accademico 2026/2027

# Indice


1 Che cos'è l'intelligenza artificiale ..... 5
1.0.1 The imitation gaming ..... 5
1.0.2 Artificial Intelligence ..... 5
1.0.3 AI oggi ..... 5
2 Che cos'è il machine learning ..... 5
2.0.1 Tre famiglie principali ..... 6
2.1 Agenti intelligenti ..... 6
2.1.1 Come dovrebbe agire un Agente intelligente? ..... 7
2.1.2 A popular framework: Markov Decision Process ..... 7
2.1.2.1 Frozen Lake ..... 7
3 Risoluzione dei problemi mediante ricerca ..... 8
3.0.1 Esempio : Vacuum-cleaner world ..... 8
3.0.2 Esempio di agent program ..... 9
3.1 Razionalità ..... 9
3.1.1 PEAS ..... 10
3.1.2 Tipi di problemi ..... 10
3.2 Problem solving agents ..... 10
3.2.0.1 Esempio: Viaggio in Romania ..... 11
3.2.0.2 Esempio: Vacuum world state soace graph ..... 12
3.2.0.3 Esempio: The 8-Puzzle ..... 13
3.2.0.4 Esempio: Three Search ..... 13
3.3 Implementazione: Stati vs Nodi ..... 14
3.3.1 Strategie di ricerca ..... 15
4 Uninformed search strategies ..... 16
4.1 Breadth-first search ..... 16
4.1.1 Espandere il nodo meno profondo ..... 16
4.1.2 Proprietà ..... 17
4.2 Uniform cost search ..... 17
4.2.1 Implementazione ..... 17
4.2.2 Proprietà ..... 17
4.2.3 Ottimalità di UCS ..... 17
4.3 Depth-first search ..... 18
4.3.1 Implementazione ..... 18
4.3.1.1 Implementazione ricorsiva ..... 19
4.3.2 Proprietà ..... 19
4.3.3 Variante: Iterative deepening search ..... 19
4.3.3.1 Proprietà ..... 19
4.4 BSF vs IDS ..... 20
4.5 Riassunto delle Proprietà ..... 20
5 Informed search strategies ..... 21
5.1 Best-First search ..... 21
5.1.1 Greedy search ..... 21
5.1.1.1 Greedy search esempio: ..... 21
5.1.1.2 Proprietà: ..... 22
5.1.2 $A^*$ search ..... 22
5.1.2.1 $A^*$ search esempio: ..... 22
5.1.2.2 Proprietà di $A^*$ ..... 23
5.2 Definizione di euristica ..... 23
5.2.1 Euristica consistente ..... 23
5.2.2 Euristica ammissibile vs Euristica consistente ..... 23
5.2.3 Euristica ammissibile ..... 24

5.2.4 Esempi tipici di ricerca: ... 24
5.2.5 Problemi rilassati ... 25
5.2.6 Funzioni euristiche e algoritmi di ricerca ... 25
6 Local Search ... 26
6.1 Iterative improvement algorithms ... 26
6.1.1 Esempio: Travelling Salespersom Problem ... 26
6.1.2 Esempio: n-queens ... 27
6.2 Hill-climbing (gradient ascent/descent) ... 27
6.3 Simulated Annealing ... 28
6.3.1 Proprietà del simulated annealing ... 29
6.4 Local beam search ... 30
6.5 Genetic algorithms ... 30
7 Local Search In Continuous Space ... 31
7.0.1 Example: Placing airports in Romania ... 31
7.0.1.1 Come possiamo affrontare questo problema? ... 31
7.1 Local Gradient ... 33
7.2 Newton-Raphson ... 33
7.2.1 Semplice applicazione ... 33
7.3 Interpretazione geometrica del metodo di Newton ... 35
7.4 Applicazioni con il gradiente ... 35
7.4.1 Empirical Gradient ... 35
8 Constraint Satisfaction Problem ... 36
8.1 Standard Search Problem ... 36
8.2 Constraint Satisfaction Problems ... 36
8.2.1 CSP Example : Map-Coloring ... 36
8.2.2 Example: N-Queens ... 37
8.2.3 N-Queens: alternative formulation ... 37
8.3 Constraint graph ... 38
8.3.1 Example: Crossword ... 38
8.3.1.1 Non binary constraints ... 39
8.3.1.2 Hypergraphs and Dual Graphs ... 39
8.4 Una prospettiva più ampia del CSP ... 39
8.4.1 Graphical Models ... 39
8.5 Mathematical formulation of Constraint Networks ... 40
8.5.1 Solutions and partial solutions ... 40
8.6 Standard search formulation for CSP ... 41
8.7 Backtracking search ... 41
8.7.1 Pseudo codice ... 42
8.8 Improving Backtracking ... 43
8.8.1 Ordering Variables: Minimum Remaining Values ... 43
8.8.2 Ordering Variables: Degree heuristic ... 43
8.8.3 Value ordering: Least Constraining Value ... 43
8.9 Inference: make hidden constraints explicit ... 45
8.9.1 Trade-off between inference and search ... 45
8.9.2 Backtrack Free Search ... 45
8.10 Look-Ahead ... 45
8.10.1 Costi e benefici ... 46
8.10.2 Forward Checking ... 46
8.10.3 Forward Checking: esempio sulla mappa ... 46
8.10.4 Forward Checking: Graph Colouring Example ... 47
8.10.5 Complexity of Forward Checking ... 47
8.11 Arc Consistency Look-Ahead ... 48
8.11.1 The power of AC: Sudoku ... 48

8.11.2 Arc Consistency Look-Ahead Complexity ..... 49
8.11.3 Example of AC Look-Ahead ..... 49
8.12 Enforcing Arc Consistency for Networks ..... 51
8.12.1 Insights on Arc Consistency ..... 51
8.13 Exploiting problem structure: Cycle Cutset ..... 51
8.13.1 Example: Australia map colouring ..... 52
8.14 Esercizi finali ..... 53
8.14.1 AC Look-Ahead sul Graph Colouring Example ..... 53
8.14.2 Cycle Cutset sul Graph Colouring Example ..... 53

# 1 Che cos'è l'intelligenza artificiale

- Nasce negli anni 50 con Alan Turing con il **Turing Test**
- 1996 **Deep Blue** per giocare a scacchi
- 2026 **Generative AI**

### 1.0.1 The imitation gaming

«Propongo di prendere in considerazione la domanda: «Le macchine possono pensare?». Si dovrebbe iniziare definendo il significato dei termini «macchina» e «pensare». Le definizioni potrebbero essere formulate in modo da riflettere, per quanto possibile, l'uso comune delle parole; tuttavia, questo atteggiamento è pericoloso. Se il significato delle parole «macchina» e «pensare» deve essere ricavato esaminando il modo in cui vengono comunemente usate, è difficile sottrarsi alla conclusione che.. «»

### 1.0.2 Artificial Intelligence

Nel 1956 fecero un progetto per capire se si potevano sviluppare dei sistemi per risolvere problemi più complessi in ambito umano e qui si usa per la prima volta il termine «Artificial Intelligence»

### 1.0.3 AI oggi

Robocup 2026



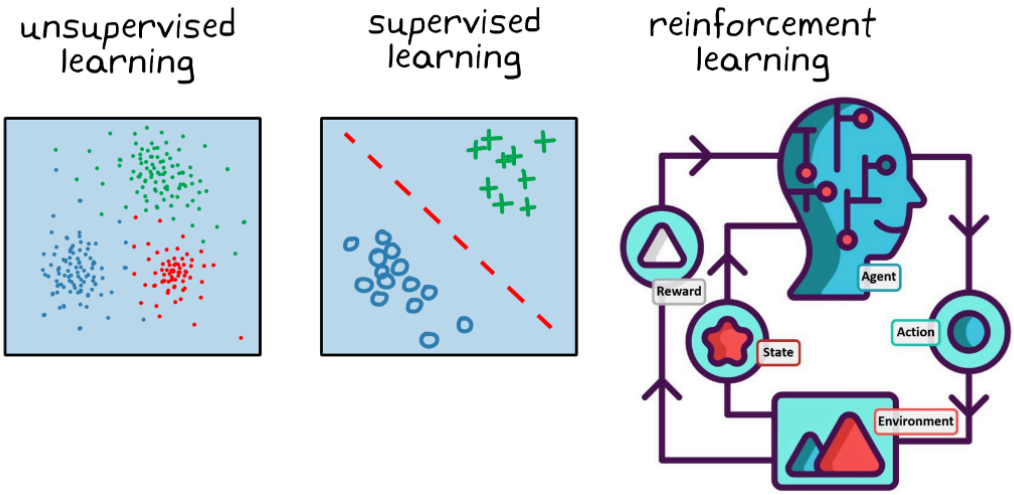

# 2 Che cos'è il machine learning

La programmazione permette ai calcolatori di risolvere task, mentre il machine learning insegna ai calcolatori come risolvere gli stessi medesimi problemi ma in modo autonomo.

### Perche il machine learning è tanto usato?

Grazie alla raccolta dei dati negli ultimi anni usare tecniche di ML per risolvere problemi è diventato molto più facile.

### 2.0.1 Tre famiglie principali



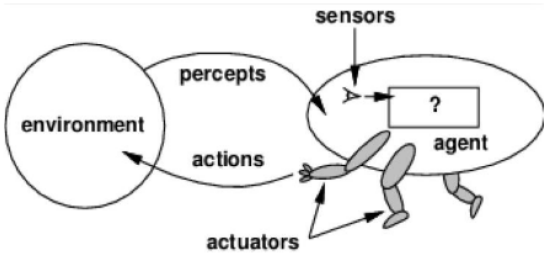

- **Non supervisionato** : non diamo informazioni al calcolatore se non solo la rappresentazione dei dati, la macchina decide in che modo suddividere i gruppi
- **Supervisionato** : diamo informazioni al calcolatore definendo noi in che modo descriminare i dati
- **Reinforcment learning** : un agente compie azioni nel mondo con un meccanismo di riccompense

## 2.1 Agenti intelligenti

Nella comunità che studia i sistemi ad agenti, è sorprendentemente difficile rispondere alla domanda **"che cos'è un agente?"**, proprio come nell'IA è difficile definire in modo universalmente accettato "che cos'è l'intelligenza?".

Il termine agent viene usato molto, anche da ricercatori di aree molto vicine, ma non esiste una definizione unica condivisa. Questo non è necessariamente un problema, perché si possono comunque costruire applicazioni utili anche senza accordarsi perfettamente sulla terminologia.

Però c'è il rischio che agent diventi una parola vaga, usata in modo improprio e quindi fonte di confusione. Per questo gli autori ritengono utile discutere il significato del termine e distinguono due modi principali di intendere "agente": uno debole, relativamente poco controverso, e uno più forte, più discutibile.




Russel e Norvig: AI a modern approach

Un agente è dotato di 3 caratteristiche:

- Analizza l'ambiente
- Agisce nell'ambiente
- Ha sempre un obiettivo

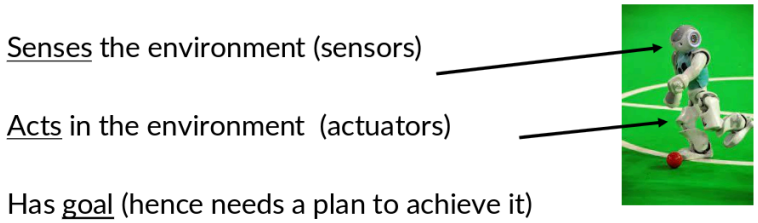
### 2.1.1 Come dovrebbe agire un Agente intelligente?

**1) Misurazione delle performance**

L'agente valuta se l'ambiente è in grado di far raggiungere all'agente l'obiettivo

**2) Razionalità**

L'agente sceglie sempre di massimizzare il valore che lo porta al raggiungimento dell'obiettivo




### 2.1.2 A popular framework: Markov Decision Process

Un esempio classico di agente è **Markov Decision Process** dove abbiamo:

- **state**: Rappresentazione dell'ambiente
- **action**: Come l'agente può modificare l'ambiente
- **transition model**: Gli effetti che hanno le azione sull'ambiente

$$T : \text{state, action} \rightarrow \text{next state}$$

- **reward**: Ricompensa immediata in risposta alla azione

$$R : \text{state, action,(next state} \rightarrow \text{real number}$$

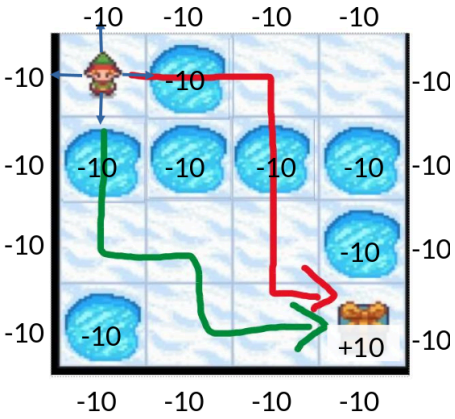
- **policy**: scelta delle azioni per massimare la ricompensa nel lungo termine

$$\pi : (\text{state}) \rightarrow \text{action}$$

#### 2.1.2.1 Frozen Lake



Prendimao in considerazione gli elementi principali della catena di Markov:



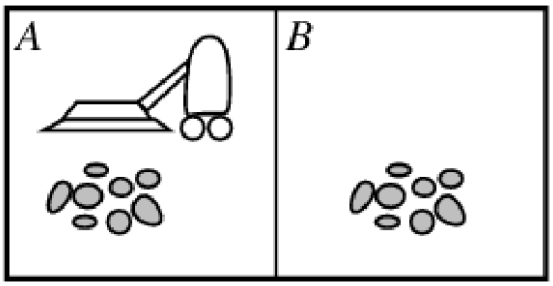

# 3 Risoluzione dei problemi mediante ricerca

Con il termine **agente** ci possiamo riferire a: umani, robot, softbot, termostati ecc..
L' **agent funtion** associa a ogni possibile sequenza di percezioni un'azione da eseguire:

$$f : P^* \to A$$

L'**agent program** invece viene eseguito sul calcolatore per produrre $f$

### 3.0.1 Esempio : Vacuum-cleaner world




- Perceptions: Posizione nello spazione e il contenuto [A, Dirty]
- Actions: left, right, Suck, NoOp

| Percept sequence | Action |
|---|---|
| [*A*, *Clean*] | *Right* |
| [*A*, *Dirty*] | *Suck* |
| [*B*, *Clean*] | *Left* |
| [*B*, *Dirty*] | *Suck* |
| [*A*, *Clean*], [*A*, *Clean*] | *Right* |
| [*A*, *Clean*], [*A*, *Dirty*] | *Suck* |
| ⋮ | ⋮ |

**Domanda**

se un agent ha $|P|$ possibili perceptions, quante entries avrà l'agent function dopo $T$ passi?

**Risposta**

$$\sum_{t=1}^{T} |P|^t$$

#### 3.0.2 Esempio di agent program

function **Reflex-Vacuum-Agent([location, status])** return an action:

```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

## 3.1 Razionalità

La misura di prestazione fissa valuta la sequenza dell'ambiente.

- è possibile pulire ogni punto dell'area in tempo $T$?
- un punto per ogni casella pulita ad ogni passo $T$, meno un punto per ogni mossa?
- penalizzare per $>$ K caselle sporche?

Un **rational agent** sceglierà qualsiasi azione che massimizzi il valore atteso della performance data la sequenza di percezioni acquisita fino a quel momento

Ricordiamo però che:

**Rational $\neq$ omniscent**

- percezioni possono non fornire tutte le informazioni rilevanti

**Rational $\neq$ clairvoyant**

- le azioni compiute possono non essere quelle che ci saremmo aspettati

**Rational $\neq$ successful**

- poichè rational $\Rightarrow$ exploration, leraning, autonomy

**Esercizio**

Considera il seguente ambiente:

- Ci sono **tre stanze (A, B, C)** e **due robot (r1, r2)**.
- **r1** può pattugliare le stanze **A e B**.
- **r2** può pattugliare le stanze **B e C**.
- **r1** parte dalla stanza **A**, mentre **r2** parte dalla stanza **C**.
- Il **tempo di spostamento tra le stanze è pari a zero**.

**Misura di prestazione:** minimizzare la somma dell'**idleness media** di tutte le stanze.

**Idleness media** = somma degli intervalli di tempo durante i quali la stanza **non viene visitata da nessun robot** / intervallo di tempo totale.

Quale sarebbe un **comportamento razionale** per questo ambiente?

### 3.1.1 PEAS

Per progettare un **rational agent** dobbiamo specificare il **task environment**.
Considerando un esempio: the task of designing an automated taxi:

- **performance measure** → safety, destination, profits, legality, comfort …
- **environment** → city street/freeways, traffic, pedestrains, weather …
- **actuators** → steering, accelerator, brake, horn, speaker/display …
- **sensors** → video, accelerometers, gauges, engine sensors, keyboard, GPS …

**Il tipo di ambiente determina in larga misura la progettazione dell'agente**

| | Crossword | Robo-selector | Poker | Taxi |
|---|---|---|---|---|
| Observable?? | Yes | Partly | Partly | Partly |
| Deterministic?? | Yes | No | No | No |
| Episodic?? | No | Yes | No | No |
| Static?? | Yes | No | Yes | No |
| Discrete?? | Yes | No | Yes | No |
| Single-agent?? | Yes | Yes | No | No |

Poichè il mondo reale è ovviamente parizalmente osservabile, stocastico, sequenziale, dinamico, continuo e multi agentico

### 3.1.2 Tipi di problemi

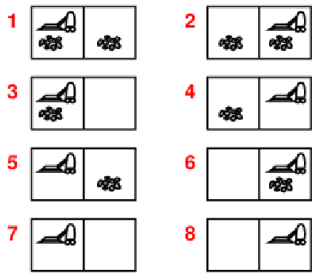
- **Deterministico e completamente osservabile → problema a stato singolo**
  - L'agente sa esattamente in quale stato si troverà.
  - La soluzione è una **sequenza di azioni**.
- **Non osservabile → problema conformante**
  - L'agente può non avere alcuna informazione sullo stato in cui si trova.
  - La soluzione, se esiste, è una **sequenza di azioni**.
- **Non deterministico e/o parzialmente osservabile → problema di contingenza**
  - Le percezioni forniscono nuove informazioni sullo stato corrente.
  - La soluzione è un **piano contingente** oppure una **policy**.
  - Spesso ricerca ed esecuzione vengono alternate.
- **Spazio degli stati sconosciuto → problema di esplorazione** (**online**)

**Esempio: Vacuum world**

Single-state, start in #5. Solution??
[*Right*, *Suck*]

Conformant
start in $\{1, 2, 3, 4, 5, 6, 7, 8\}$
e.g., *Right* goes to $\{2, 4, 6, 8\}$. Solution??
[*Right*, *Suck*, *Left*, *Suck*]

Contingency, start in #5
Murphy's Law: *Suck* can dirty a clean carpet
Local sensing: dirt, location only.
Solution??
[*Right*, if *dirt* then *Suck*]




## 3.2 Problem solving agents

Derivazione di un agente generale: **agenti basati su obiettivi**

- formulano un obiettivo e un problema dato uno stato corrente
- cercano una soluzione
- eseguono la soluzione **ingnorando** le percezioni

Questo a volte viene anche chiamato: **offline problem solving** → solutions executed «eyes closed»

```
function Simple-Problem-Solving-Agent(percept) returns an action
    static: seq, an action sequence, initially empty
            state, some description of the current world state
            goal, a goal, initially null
            problem, a problem formulation

    state ← Update-State(state, percept)

    if seq is empty then
        goal ← Formulate-Goal(state)
        problem ← Formulate-Problem(state, goal)
        seq ← Search(problem)

    action ← First(seq)
    seq ← Rest(seq)

    return action
```

#### 3.2.0.1 Esempio: Viaggio in Romania

**Esempio**

On holiday in Romania; currently in Arad.
Flight leaves tomorrow from Bucharest

**Formulate goal**

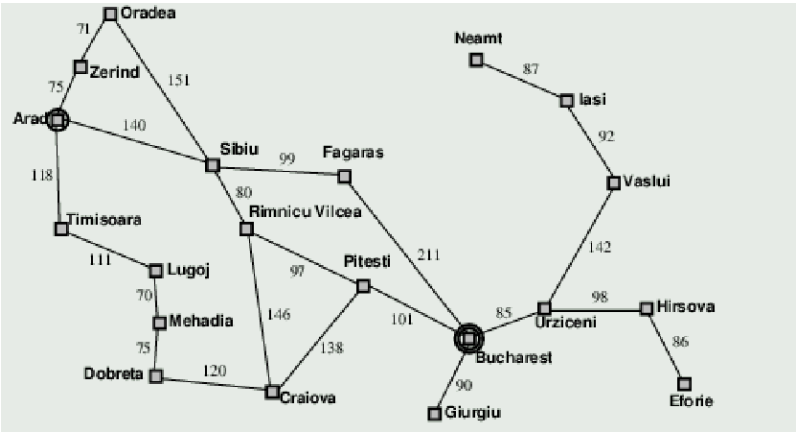
- be in Bucharest

**Formulate problem**

- **states**: various cities
- **actions**: drive between cities

**Find Solution**:

- sequence of cities e.g: Arad, Sibiu, Fagaras, Bucharest




Un problema è definito da quattro elementi:

- stato iniziale
  - ad esempio, «ad Arad»
- funzione successore $S(x)$ = insieme di coppie azione-stato
  - ad esempio, $S(A) = \{< \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} >, ...\}$
- test dell'obiettivo, può essere

- esplicito, ad esempio, $x =$ a Bucharest
- implicito, ad esempio, NoDirt(x)

- costo del percorso (additivo)
  - ad esempio, somma delle distanze, numero di azioni eseguite, ecc.

$c(x, a, y)$ è il costo del passo, assunto essere $\geq 0$

**Una soluzione è una sequenza di azioni che porta dallo stato iniziale a uno stato obiettivo.**

Il mondo reale è assurdamente complesso poichè lo spazio degli stati deve essere astratto per risolvere il problema

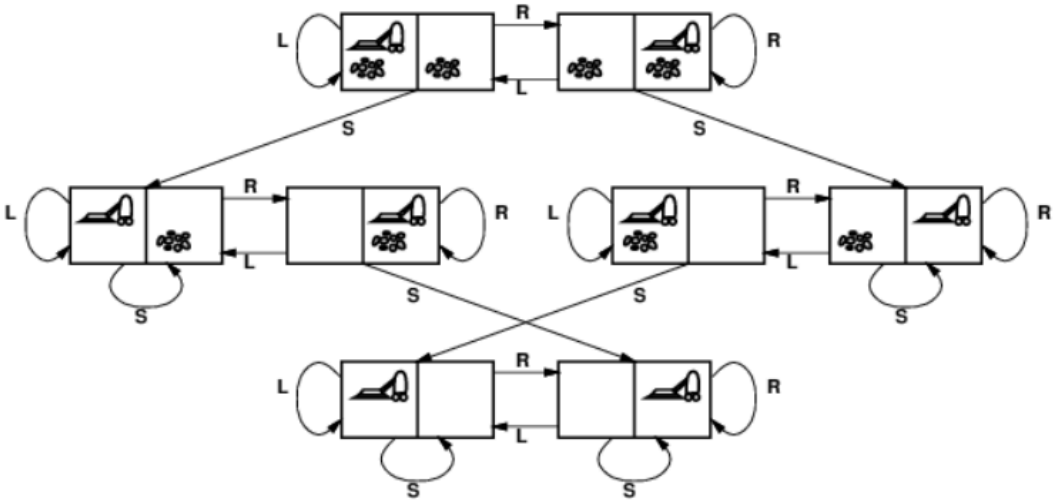
- stato (astratto) = insieme di stati reali
- azione (astratta) = combinazione complessa di azioni reali

ad esempio, «Arad -> Zerind» rappresenta un insieme complesso di possibili percorsi, deviazioni, soste, ecc.
Per garantire la realizzabilità, qualsiasi stato reale «ad Arad» deve poter raggiungere qualche stato reale «a Zerind»

- soluzione (astratta) = insieme di percorsi reali che sono soluzioni nel mondo reale

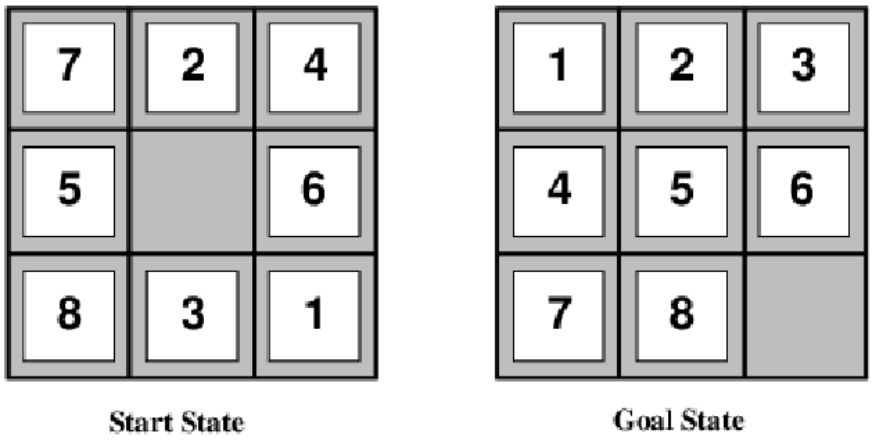
**Ogni azione astratta dovrebbe essere «più semplice» del problema originale!**

#### 3.2.0.2 Esempio: Vacuum world state soace graph




- **states** = screte dirt and robot location (ingore dirt amounts)
- **actions** = Left, Right, Suck, NoOp
- **goal test** = no dirt
- **path cost** = 1 per action (0 for NoOp)

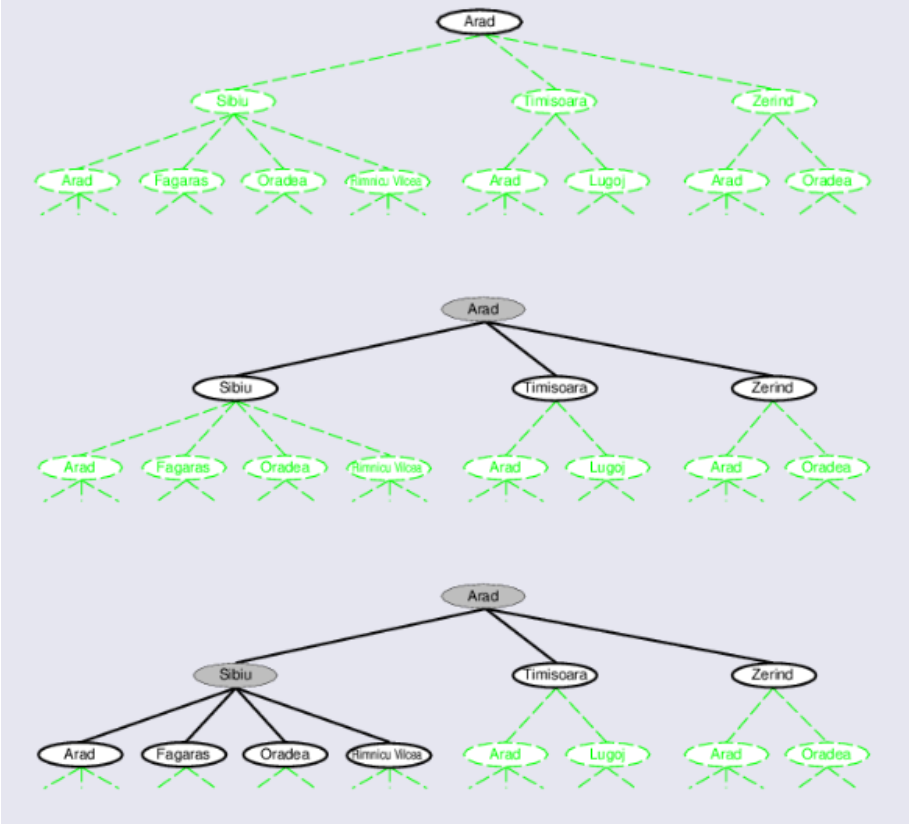
#### 3.2.0.3 Esempio: The 8-Puzzle




- **states** = integer locations of tiles (ignore intermediate potisions)
- **actions** = move blank left, right, up, down
- **goal test** = given goal state
- **path cost** = 1 per move

[Nota: soluzioni ottimali di n-Puzzle ∈ NP-hard]

#### 3.2.0.4 Esempio: Three Search

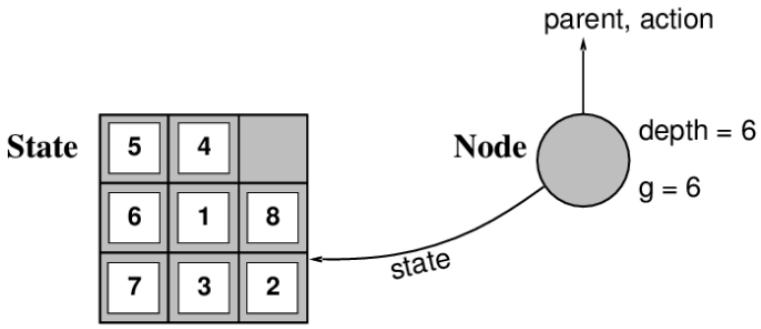


### 3.3 Implementazione: Stati vs Nodi

Uno **stato** è la rappresentazione di una configurazione fisica

Un **nodo** invece è una struttura dati composta da parti di uno search tree

- parent
- action
- children
- depth
- path cost (i.e, g(x))




La funzione **expand** crea nuovi nodi, usano il **SuccessorFn** del problema per creare il corrispondente stato

**Tree Search**

```
function Tree-Search(problem, frontier) returns a solution, or failure
frontier <- Insert(Make-Node(problem.Initial-State))
while not IsEmpty(frontier) do
  node <- Pop(frontier)
  if problem.Goal-Test(node.State) then
    return node
  frontier <- InsertAll(Expand(node, problem))
end loop
return failure
```

**Expand**

```
function Expand(node, problem) returns a set of nodes
successors <- the empty set
for each action, result in Successor-Fn(problem, node.State) do
  s <- a new Node

  s.Parent-Node <- node
  s.Action <- action
  s.State <- result
  s.Path-Cost <- node.Path-Cost + Step-Cost(node.State, action, result)
  s.Depth <- node.Depth + 1

  add s to successors
return successors
```

### 3.3.1 Strategie di ricerca

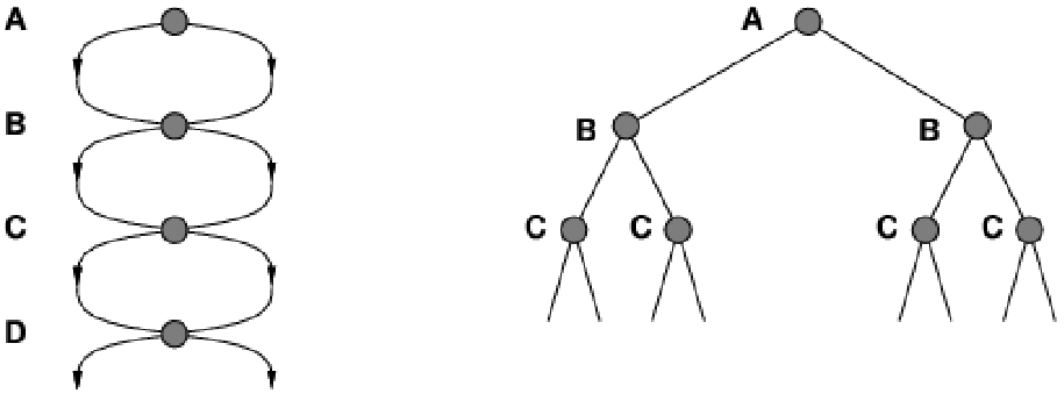
Una strategia è definita scegliendo l'ordine di espansione dei nodi. Le strategie vengono valutate secondo le seguenti dimensioni:

- **completezza** — permette di trovare una soluzione, se ne esiste una?
- **complessità temporale** — numero di nodi generati/espansi
- **complessità spaziale** — numero massimo di nodi mantenuti in memoria
- **ottimalità** — permette di trovare una soluzione di costo minimo?

La complessità temporale e spaziale vengono misurate in funzione di:

- $b$ — fattore di ramificazione massimo dell'albero di ricerca
- $d$ — profondità della soluzione di costo minimo
- $m$ — profondità massima dello spazio degli stati (può essere $\infty$)

**Fallire nel rilevare stati ripetuti può trasformare il problema da lineare a non lineare!**




**Definiamo allora il problema come:**

```
function Graph-Search(problem, frontier) returns a solution, or failure
explored <- an empty set
frontier <- Insert(Make-Node(problem.Initial-State))

while not IsEmpty(frontier) do
  node <- Pop(frontier)
  if problem.Goal-Test(node.State) then
    return node
  if node.State is not in explored then
    add node.State to explored
    frontier <- InsertAll(Expand(node, problem))
  end if
end loop

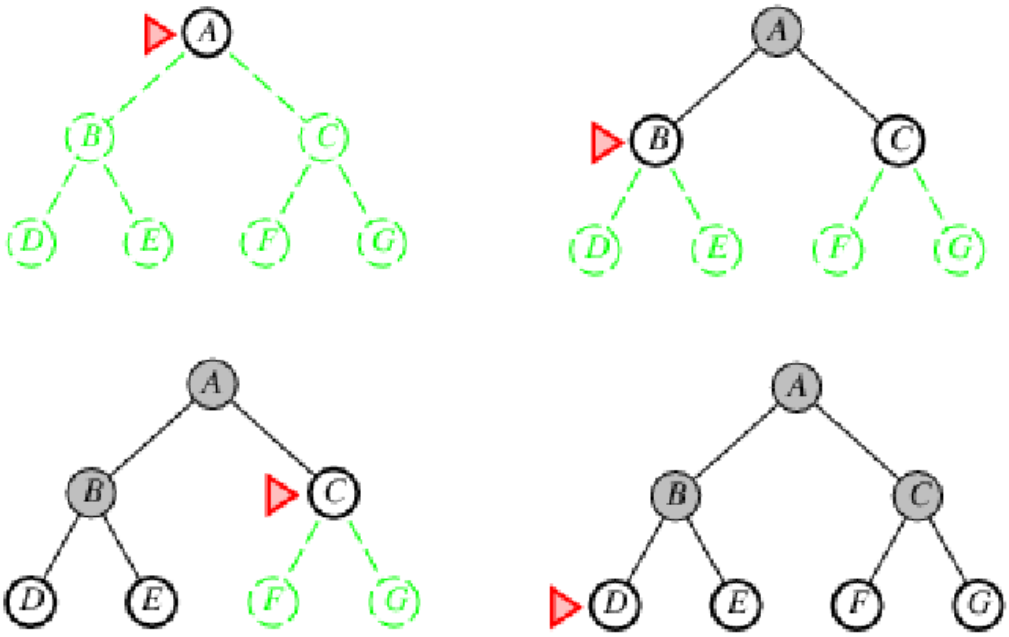
return failure
```

# 4 Uninformed search strategies

## 4.1 Breadth-first search

### 4.1.1 Espandere il nodo meno profondo

La **frontier** ovvero la frontiera (nodi che non sono ancora stati scoperti) sono rappresentati come code FIFO, il nuovo sucessore viene messo alla fine.




```
function BFS(problem) returns a solution, or failure
    node ← node with State=problem.Initial-State, Path-Cost=0
    if problem.Goal-Test(node.State) then return node
    explored ← empty set
    frontier ← FIFO queue with node as the only element

    loop do
        if frontier is empty then return failure
        node ← Pop(frontier)
        add node.State to explored

        for each action in problem.Actions(node.State) do
            child ← Child-Node(problem, node, action)

            if child.State is not in explored or frontier then
                if problem.Goal-Test(child.State) then return child
                frontier ← Insert(child)
            end if
        end for
    end loop
```

### 4.1.2 Proprietà

**Completo?** si. se b è finito (numero nodi)
**Tempo?** $b + b^2 + b^3 + ... + b^d = \mathrm{O}(b^d)$
**spazio?** $\mathrm{O}(b^d)$ (ogni nodo in memoria)
**ottimale?** si se il costo rimane costante ad ogni passagio (in generale non lo è)

**Spazio**

E' il problema principale di BFS, poiche può generare nodi a 100MB/sec in 24h può raggiungere 8640GB

## 4.2 Uniform cost search

Espandere il nodo non espanso a costo minimo (i.e minimum step cost), una versione di Dijkstra specializzata in ricerca

### 4.2.1 Implementazione

**frontier** = coda ordinata per path cost, dal più basso al più alto (equivalente a BFS se il costo ad ogni step è costante)

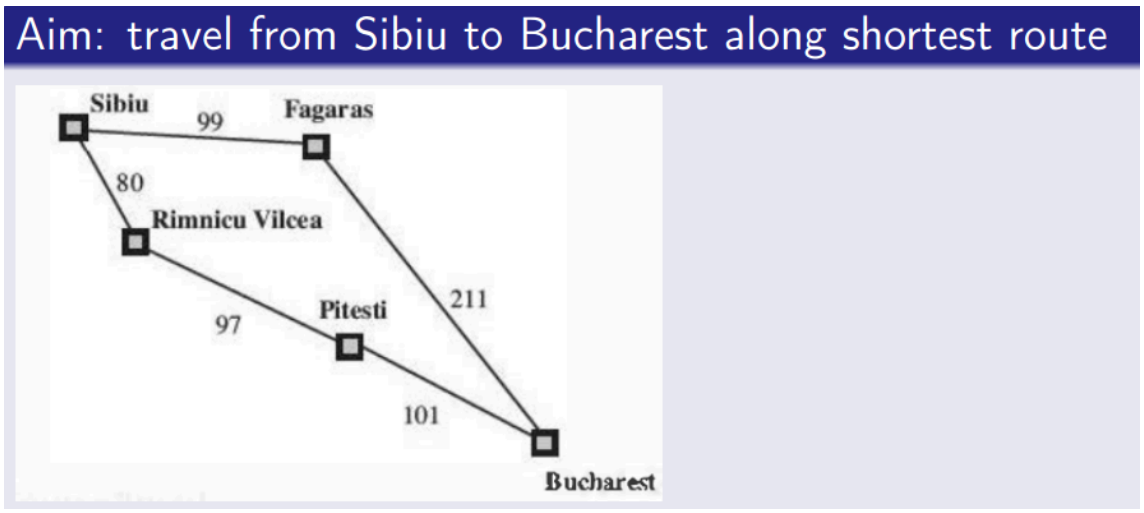
### 4.2.2 Proprietà

**Completo?** si. se il costo ad ogni step è $> \varepsilon$
**Tempo?** # di nodi con $g \leq$ del costo ottimale, $O\left(b^{1+\left\lfloor \frac{C^*}{\varepsilon} \right\rfloor}\right)$ dove $C*$ è il costo ottimale
**spazio?** # di nodi con $g \leq$ del costo ottimale, $O\left(b^{1+\left\lfloor \frac{C^*}{\varepsilon} \right\rfloor}\right)$ dove $C*$ è il costo ottimale **ottimale?** si, i nodi si espandono in ordine di $g(n)$

**due modifiche a BFS per ottener una UCS**

1) Goal test quando espandi, non quando generi
2) Sostituire un nodo nella frontier se trovi un percorso meno costoso

### 4.2.3 Ottimalità di UCS

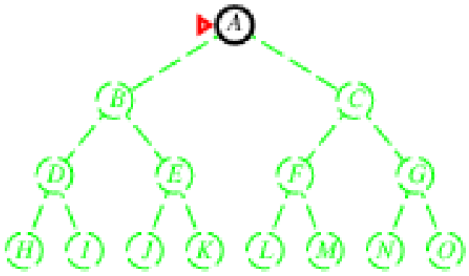
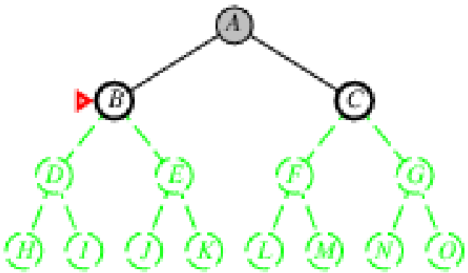
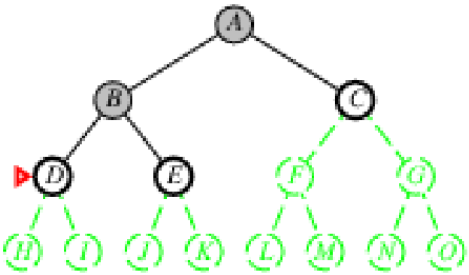
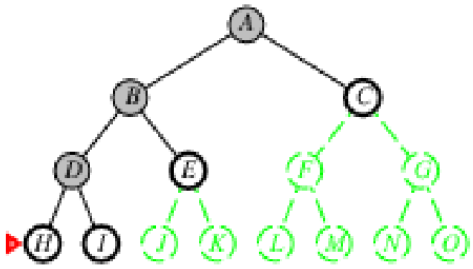
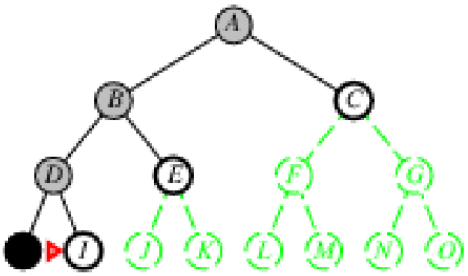
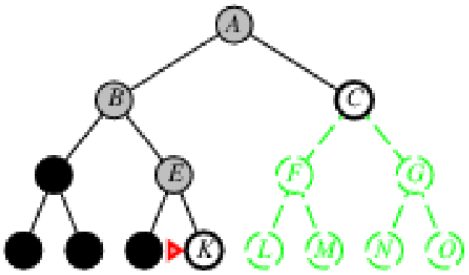
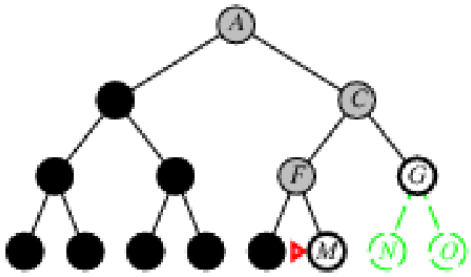



Consideriamo la mappa in figura e UCS, ci dimostra che per l'appunto:
1) Goal test quando espandi, non quando generi
2) Sostituire un nodo nella frontier se trovi un percorso meno costoso

## 4.3 Depth-first search

### 4.3.1 Implementazione

**frontier** = coda LIFO

#### 4.3.1.1 Implementazione ricorsiva

```
function DLS(problem, limit) returns soln/fail/cutoff
    R-DLS(Make-Node(problem.Initial-State), problem, limit)

function R-DLS(node, problem, limit) returns soln/fail/cutoff
    if problem.Goal-Test(node.State) then return node
    else if limit = 0 then return cutoff
    else
        cutoff-occurred? ← false

        for each action in problem.Actions(node.State) do
            child ← Child-Node(problem, node, action)
            result ← R-DLS(child, problem, limit-1)

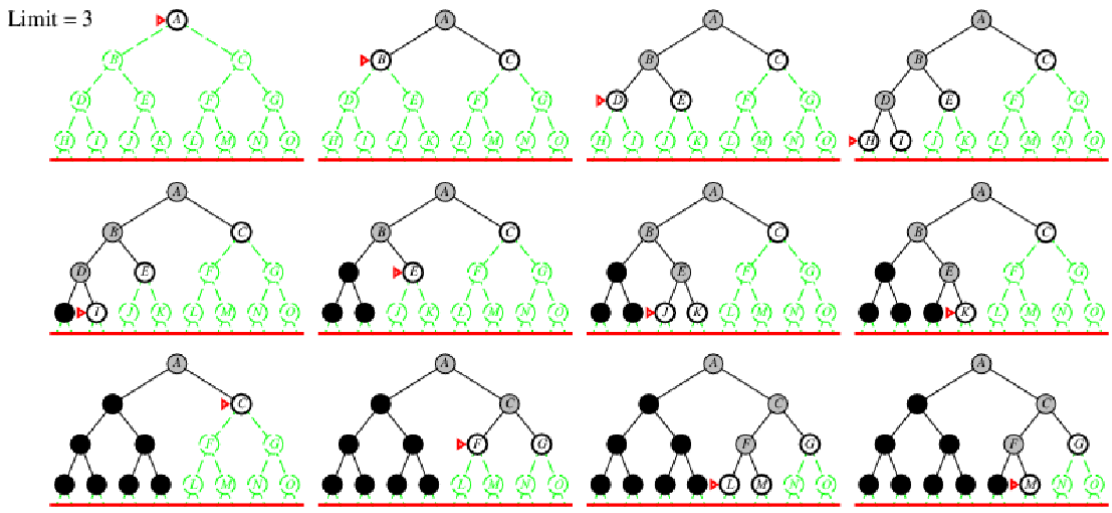
            if result = cutoff then cutoff-occurred? ← true
            else if result ≠ failure then return result
        end for

        if cutoff-occurred? then return cutoff
        else return failure
    end else
```

### 4.3.2 Proprietà

**Completo?** no, fallisce per spazzi con profondità infinit e in presenza di cappi. Modifiche al grafo per evitare stati ripetuti portano a rendere il sistema completo
**Tempo?** $O(b^m)$: terribile se m è più grande di d
**spazio?** $O(bm)$ spazio lineare
**ottimale?** No!

### 4.3.3 Variante: Iterative deepening search




```
function IDS(problem) returns a solution
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← DLS(problem, depth)
        if result ≠ cutoff then return result
    end
```

#### 4.3.3.1 Proprietà

**Completo?** si
**Tempo?** $db^1 + (d-1)b^2 + ... + b^d = O(b^d)$

**spazio?** $O(b^d)$
**ottimale?** si, se il costo ad ogni step è = 1

## 4.4 BSF vs IDS

Scegliamo b = 10 e d = 5, avremmo che :

$$N(\text{IDS}) = 6 + 50 + 400 + ... = 123456$$

$$N(\text{BFS}) = 10 + 100 + 1000 + ... = 111101$$

IDS ripete alcuni nodi ma il costo non diventa tanto più grande di BFS poichè la complessità è dettata dalla crescita esponenziale dei nodi

## 4.5 Riassunto delle Proprietà

Basandoci su tree-search

| Criterion | BF | UC | DF | DL | ID |
|---|---|---|---|---|---|
| Complete? | Yes* | Yes*,† | No | Yes*, if $l \geq d$ | Yes* |
| Time | $b^d$ | $b^{1+\lfloor C^*/\epsilon \rfloor}$ | $b^m$ | $b^l$ | $b^d$ |
| Space | $b^d$ | $b^{1+\lfloor C^*/\epsilon \rfloor}$ | $bm$ | $bl$ | $bd$ |
| Optimal? | Yes⋆ | Yes | No | Yes⋆, if $l \geq d$ | Yes⋆ |

*: complete if branching factor is finite
†: complete if step cost is $\geq \epsilon$
⋆: optimal if step costs are all identical

# 5 Informed search strategies

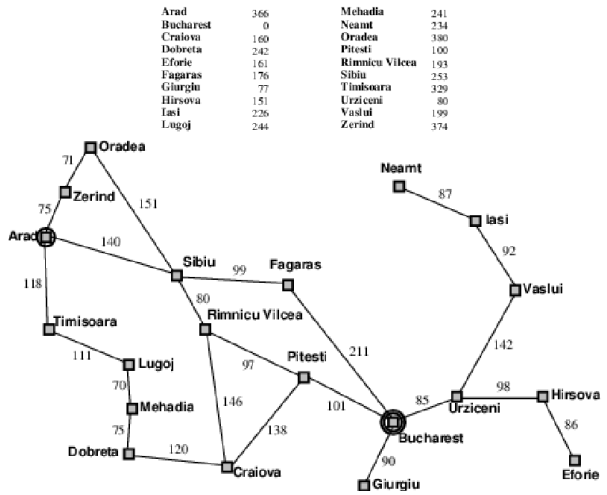
Una strategia è definita dalla scelta dell'ordine di espansione dei nodi

## 5.1 Best-First search

**Idea** = usare una funzione di valutazione per ogni nodo (stima di desiderabilità) per espandere il nodo non espanso più promettente (desidedrato)
**Implementazione** = la **frontier** è una coda ordinata descrescente in base alla desiderabilità
**Casi speciali** :

- greedy best-first search
- $A^*$ search

**Roamania with straight-line distances to Bucharest**

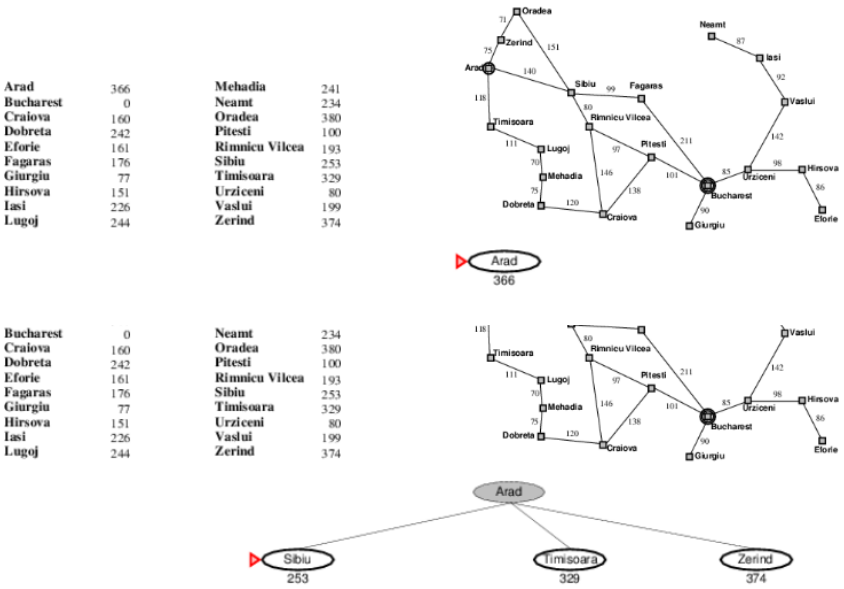



### 5.1.1 Greedy search

Sia una funzione di valutazione $h(n)$ euristica t.c Stima del costo da n all'obiettivo più vicino.
Esempio: $h_{SLD}(n)$ = straight-line distance da n a Bucharest.

Greedy search espande il nodo che si avvicina di più all'obiettivo

#### 5.1.1.1 Greedy search esempio:






#### 5.1.1.2 Proprietà:

**Completo?** No, può bloccarsi in loop (completo in spazi finiti e con check su cicli)
**Tempo?** O($b^m$) ma una buona euristica può migliorare mostrosamente
**spazio?** O($b^m$) (ogni nodo in memoria)
**ottimale?** No

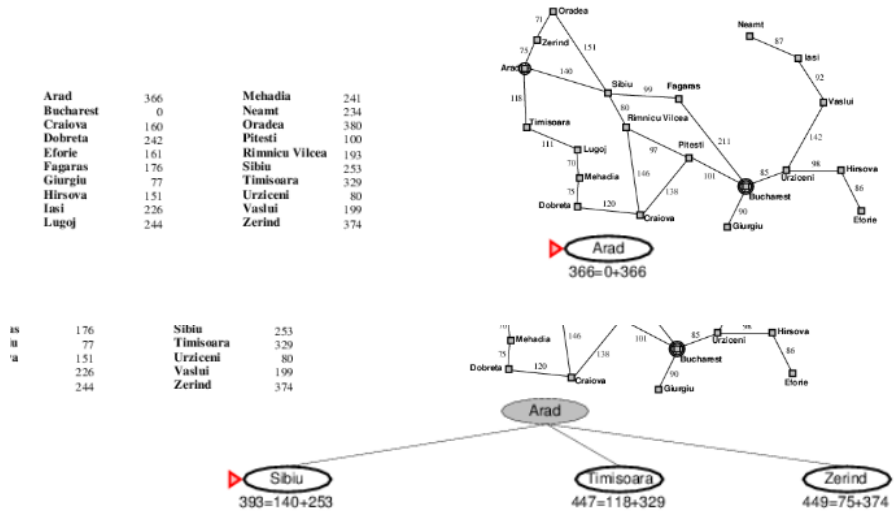
### 5.1.2 $A^*$ search

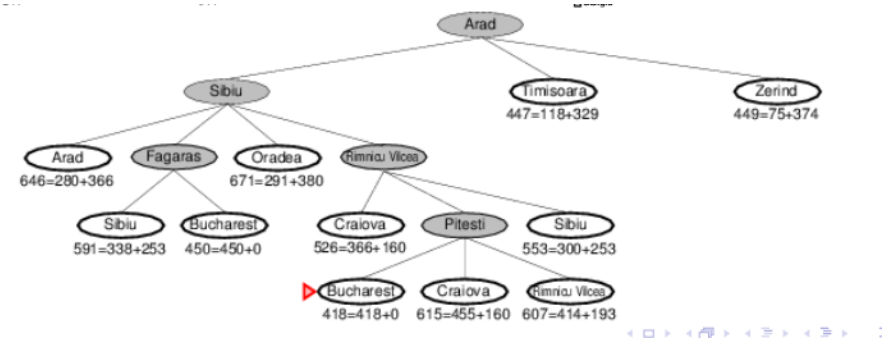
**Idea** = evitare di espandere path che hanno un costo elevato.
Sia una funzione di valutazione $f(n) = g(n) + h(n)$ con:

- $g(n)$ = costo per raggiungere nodo N
- $h(n)$ = stima del costo dall'obiettivo ad N
- $f(n)$ = stima del costo totale da N all'obiettivo

$A^*$ usa una ottima euristica:
Sia $h(n) \leq h^*(n)$ dove $h^*(n)$ è il vero costo da n (richiede che $h(n) > 0$, quindi $h(G) = 0$ per ogni obiettivo $G$)

#### 5.1.2.1 $A^*$ search esempio:






#### 5.1.2.2 Proprietà di $A^*$

**Completo?** Si, fino a quando non ci sono infiniti nodi con $f < f(g)$
**Tempo?** Esponenziale
**spazio?** Ogni nodo in memoria
**ottimale?** Si, **ma** richiede assunzioni euristiche (ammissibilità e consistenza) e search strategy (tree o graph search)

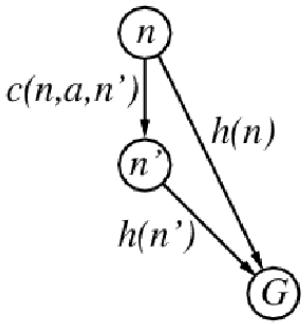
## 5.2 Definizione di euristica

### 5.2.1 Euristica consistente

Una euristica si può definire consistente se:

$$h(n) \leq c(n, a, n') + h(n')$$

1) Possiamo dimostrare che se $h$ è consistente allora $f(n)$ non è decrescente su tutto il path
2) $A^*$ espande nodi in ordine di crescita rispetto ad $f$, di conseguenza, troverà la soluzione a costo minimo.



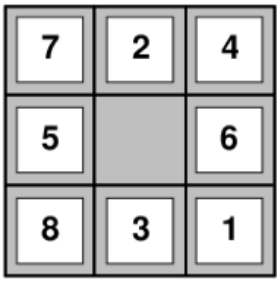

### 5.2.2 Euristica ammissibile vs Euristica consistente

**Consistenza → Ammissibile**
si può dimostrare per induzione sul percorso verso l'obiettivo.

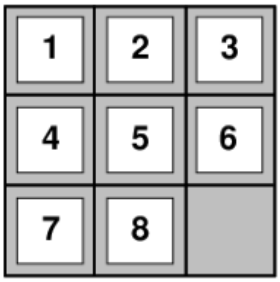
**Ammissibile ↛ Consistenza**
basti trovare un controesempio...

Tree-Search + euristica ammissibile → ottimale per $A^*$
Graph-Search + euristica ammissibile ↛ ottimale per $A^*$

È possibile scartare il percorso ottimale verso un nodo già visitato, quindi:
Graph-Search + euristica consistente → ottimale per $A^*$



Start State



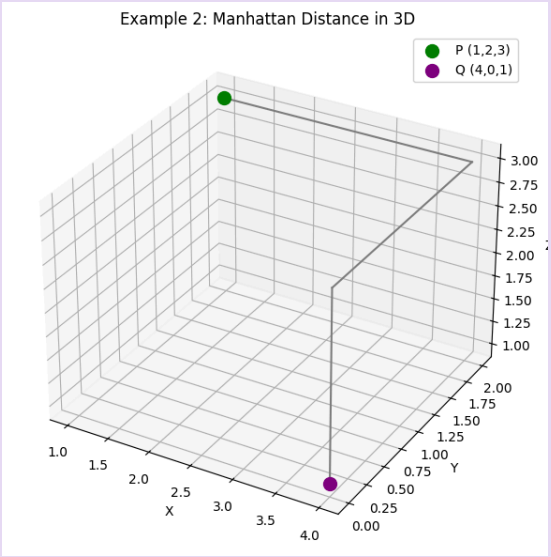
Goal State

### 5.2.3 Euristica ammissibile

Prendiamo il problema del 8-puzzle:
$h_1(n)$ = numero di tessere fuori posto $h_2(n)$ = distanza di Manhattan totale

**Distanza di Manhattan**

è una metrica che misura la distanza tra due punti sommando il valore assoluto delle differenze delle loro coordinate




$h_1(S) = 6$
$h_2(n) = 4+0+3+3+1+0+2+1 = 14$

Se $h_2(n) \geq h_1(n)$ per ogni n, allora $h_2$ domina $h_1$ ed è migliore per la ricerca

### 5.2.4 Esempi tipici di ricerca:

Per d = 14:
IDS = 3473941 nodi
$A^*(h_1) = 539$ nodi
$A^*(h_2) = 113$ nodi

Per d = 24:
IDS ≈ 54000000000 nodi
$A^*(h_1) = 39135$ nodi
$A^*(h_2) = 1641$ nodi
**Data una euristica ammissibile $h_a$ e $h_b$ :** $h(n) = \max\left(h_{a(n)}, h_{b(n)}\right)$

Possiamo quindi dire:

- Ammissibile: non sovrastima mai il costo reale minimo per raggiungere l'obiettivo.
- Consistente: rispetta una particolare relazione tra il costo stimato di due nodi vicini.

### 5.2.5 Problemi rilassati

Euristiche ammissibili possono essere ricavate dalla esatta soluzione del costo della **versione rilassata** del problema, ovvero un problema al quale sono stati eliminati alcuni vincoli rendendo il problema più semplice da risolvere

- Se le regole di 8-puzzle sono rilassate quindi una casella può essere mossa **ovunque**, allora $h_1(n)$ ci da la soluzione più rapida
- Se le regole di 8-puzzle sono rilassate quindi una casella può essere mossa in una qualsiasi casella adiacente, allora $h_2(n)$ ci da la soluzione più rapida

**Key point**

Il costo della soluzione ottimale di un **problema rilassato** non è più grande del costo della soluzione ottimale del **problema reale**

### 5.2.6 Funzioni euristiche e algoritmi di ricerca

- Le **funzioni euristiche** stimano il costo del percorso più breve per raggiungere l'obiettivo.
- Buone euristiche possono ridurre drasticamente il costo della ricerca.
- **Greedy Best-First Search**:
  - Espande il nodo con il valore $h(n)$ più basso.
  - Non è completo in generale e non garantisce una soluzione ottima.
- **A* Search**:
  - Espande il nodo con il valore $f(n) = g(n) + h(n)$ più basso.
  - È completo e ottimale sotto opportune condizioni.
  - È anche **ottimamente efficiente** (a meno dei casi di parità nella scelta dei nodi, per la ricerca in avanti).
- Le **euristiche ammissibili** possono essere ricavate dalla soluzione esatta di problemi rilassati.

# 6 Local Search

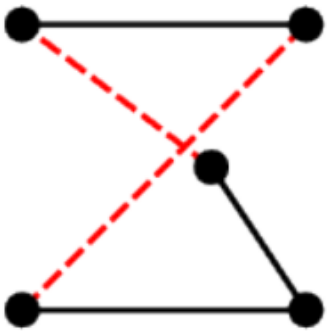
Local Search è un insieme di metodologie euristiche per trovare una soluzione ottimale di problemi difficili, discreti o non differenziabili.

## 6.1 Iterative improvement algorithms

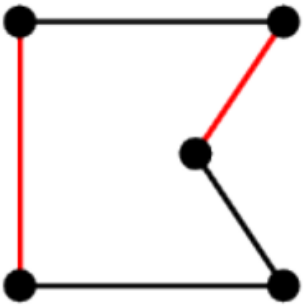
- In molti problemi di ottimizzazione il **path** diventa irrilevante → il goal è la soluzione al problema.
- Lo spazio degli stati = l'insieme delle configurazioni «complete», ovvero che a differenza della ricerca classica in spazi di stati (dove ogni nodo è un passo intermedio e gli stati iniziali sono incompleti) qui ogni stato rappresenta già una soluzione completa.
- Possiamo usare algoritmi iterativi, per esempio invece di dover tenere in memoria un albero completo di ricerca usiamo un singolo nodo per spostarci nell'albero verso la soluzione migliore.
- Adatto poiché usa spazi costanti di memoria (un singolo nodo)

### 6.1.1 Esempio: Travelling Salespersom Problem

L'algoritmo inizia con un tour completo ma non ottimale (cioè un percorso che visita tutte le città e torna all'inizio)

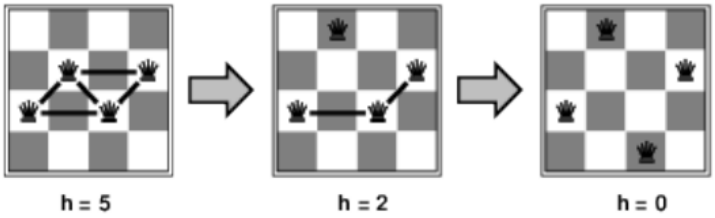


Nel grafico a sinistra, le linee tratteggiate rosse si intersecano (formano una specie di «X»). Nella geometria euclidea, due linee che si incrociano creano quasi sempre un percorso più lungo rispetto a ridisegnarle in modo da eliminare l'intersezione.
Applicando una soluzione più ottimale invece:



### 6.1.2 Esempio: n-queens

- Mettiamo n regine su una scacchiera nxn in modo che non ci siamo due regine sulla stessa riga/colonna/diagonale
- Muoviamo una regina in modo da ridurre il numero di conflitti



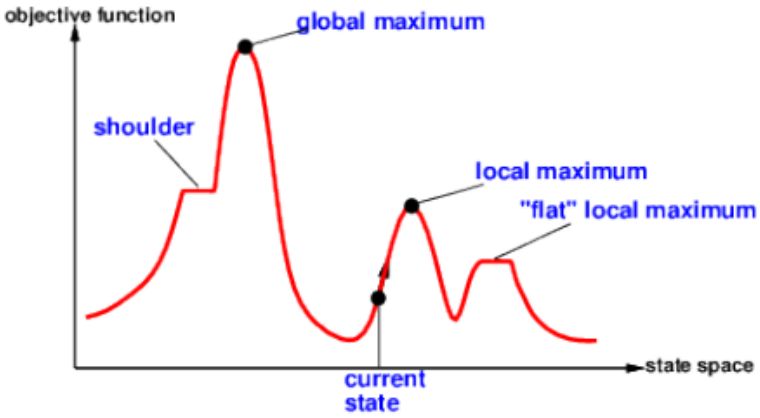

## 6.2 Hill-climbing (gradient ascent/descent)

sia:

```
function Hill-Climbing(problem) returns a state that is a local maximum
  inputs: problem, a problem
  local variables: current, a node
                   neighbor, a node

  current <- Make-Node(problem.Initial-State)
  loop do
    neighbor <- a highest-valued
```

L'algoritmo si muove alla cieca. Sa solo se sta salendo rispetto alla posizione immediatamente vicina, ma non ha memoria di dove sia stato né ha una visione d'insieme della montagna (non sa dove sia la cima più alta in assoluto)
Mostriamo ora lo **state-space** del nostro problema:




dove **shoulder** è il nostro «minimo locale», dove l'algoritmo potrebbe bloccarsi per passi molto piccoli pensando non ci sia più nessun punto di discesa verso un possibile minimo globale
Per risolvere questo problema abbiamo 2 soluzioni possibili:

- **Random-restart hill-climbing (Hill-climbing con riavvii casuali)**: Risolve il problema dei massimi locali. Se l'algoritmo si blocca, ricomincia da un punto completamente casuale dello spazio di ricerca. Ripetendo il processo, diventa «trivially complete» (cioè, con probabilità che tende a 1, prima o poi troverà il massimo globale)
- **Random sideways moves (Mosse casuali laterali)**: Permette all'algoritmo di fare passi laterali (su zone pianeggianti) per sfuggire alle spalle (shoulders), ma con il rischio di entrare in un ciclo infinito (loop) se si trova su un massimo locale piatto

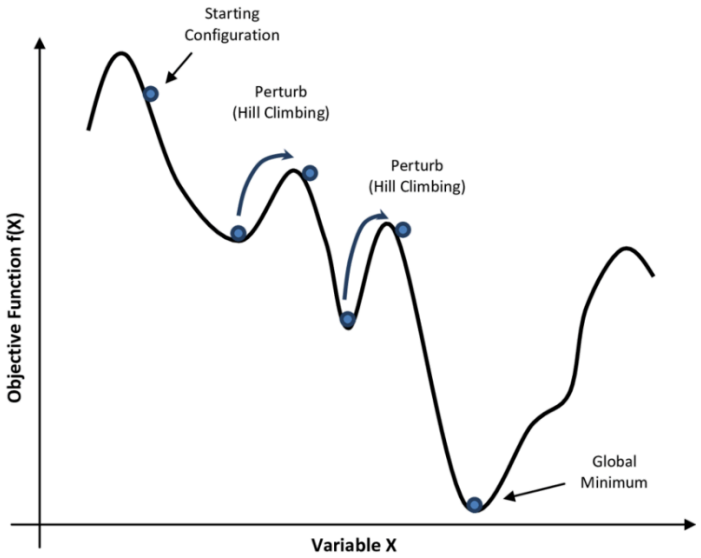
## 6.3 Simulated Annealing

**Ispirato alla meccanica statistica:** Prende il nome dal processo di tempra (annealing) dei metalli, in cui il materiale viene riscaldato e poi raffreddato lentamente per raggiungere una struttura cristallina stabile e priva di difetti.

**Accettare mosse «cattive»:** A differenza dell'Hill-Climbing (che rifiuta qualsiasi mossa che peggiori la situazione), il Simulated Annealing permette occasionalmente di fare passi «in salita» (o peggiorativi), ma diminuisce gradualmente questa frequenza con il passare del tempo

**Fasi dell'algoritmo:**

- All'inizio: Permette molte mosse casuali, consentendo all'algoritmo di saltare fuori da valli profonde e esplorare nuove zone dello spazio delle soluzioni.
- Verso la fine: La probabilità di fare mosse casuali diminuisce drasticamente, permettendo all'algoritmo di concentrarsi sul raffinamento della ricerca attorno a una buona soluzione.




**Pseudo codice:**

```
function Simulated-Annealing(problem, schedule) returns a solution state
  inputs: problem, a problem
          schedule, a mapping from time to "temperature"
  local variables: current, a node
                   next, a node
                   T, a "temperature" controlling prob. of downward steps

  current <- Make-Node(problem.Initial-State)
  for t <- 1 to infinity do
    T <- schedule(t)
    if T = 0 then return current
    next <- a randomly selected successor of current
    ΔE <- next.Value - current.Value
    if ΔE > 0 then current <- next
    else current <- next only with probability e^(ΔE / T)
  end
```

### 6.3.1 Proprietà del simulated annealing

Prendendo spunto dal sistema fisico:
A una «temperatura» $T$ fissa, la probabilità di occupare un determinato stato segue la distribuzione di Boltzmann:

$$p(x) = \alpha e^{\frac{E(x)}{kT}}$$

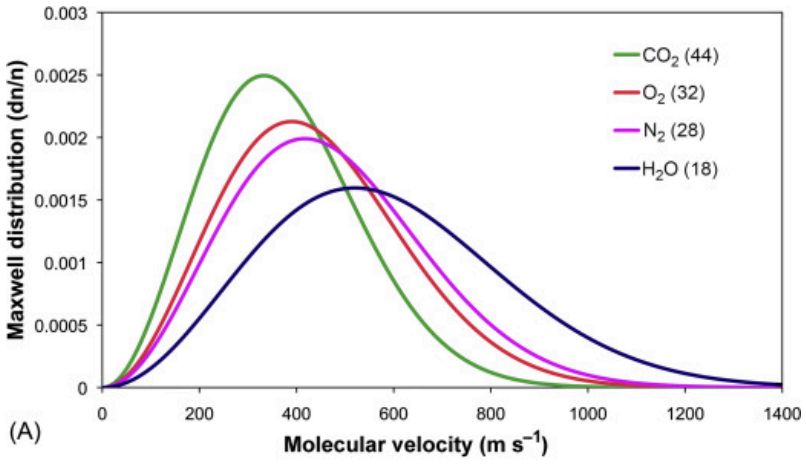
Se la temperatura $T$ viene ridotta abbastanza lentamente, il sistema è teoricamente garantito di raggiungere sempre lo stato migliore $x^*$

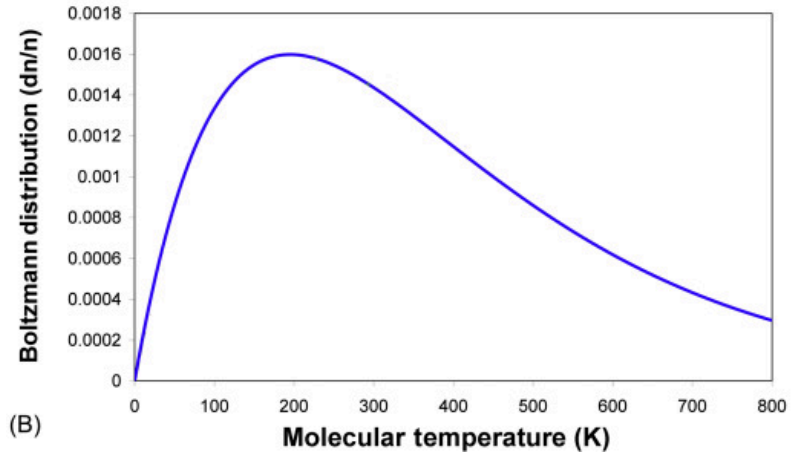
Ciò è dovuto al fatto che, per valori molto piccoli di $T$, il rapporto tra le probabilità dello stato ottimale e di un altro stato ($\frac{e^{\frac{E(x^*)}{kT}}}{e^{\frac{E(x)}{kT}}} = e^{\frac{E(x^*)-E(x)}{kT}}$) diventa enormemente superiore a 1 ($\gg 1$).

**È una garanzia davvero utile nella pratica?** In teoria la convergenza all'ottimo globale è garantita, ma sul piano pratico la risposta è spesso negativa. Dire che la temperatura deve scendere «abbastanza lentamente» significa che potrebbe servire un tempo di calcolo esponenziale, rendendo la garanzia teorica poco utile per problemi reali e complessi.

**Origini e Applicazioni**

- Sviluppato inizialmente da Metropolis et al. nel 1953 per simulare e modellare processi fisici e termodinamici.
- Trova grande applicazione nell'ottimizzazione combinatoria, come la progettazione di circuiti integrati (layout VLSI) e la gestione dei turni e dei voli nelle compagnie aeree (airline scheduling).



## 6.4 Local beam search

**Cos'è il Local Beam Search?**

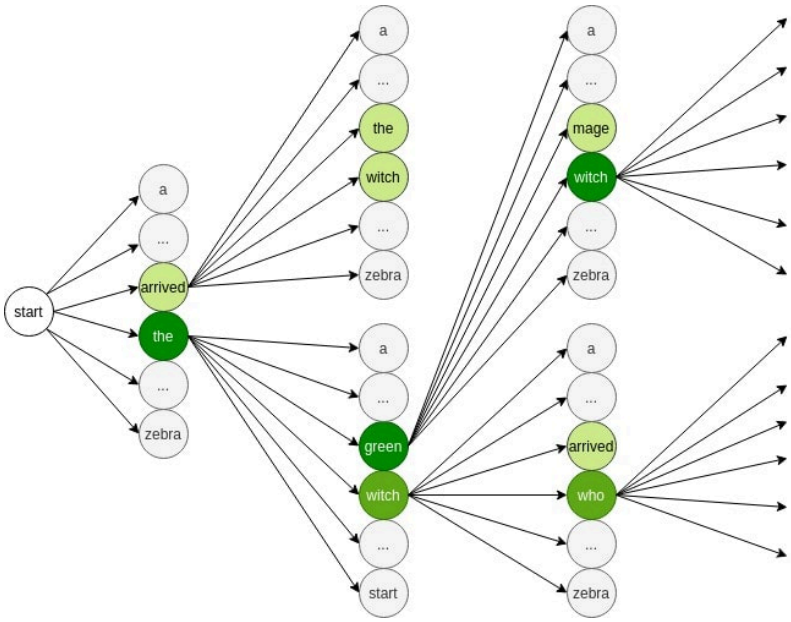
- **Idea principale:** A differenza dell'Hill-Climbing classico che mantiene un solo stato, qui si mantengono traccia di $k$ stati generici contemporaneamente. Ad ogni passo, vengono generati tutti i successori di questi $k$ stati, e tra tutti si scelgono i $k$ migliori in assoluto.
- **Non sono ricerche indipendenti:** Questa tecnica non corrisponde all'esecuzione di $k$ ricerche in parallelo separate. I vari «fasci» comunicano tra loro: le ricerche che trovano stati promettenti «reclutano» le altre, facendo sì che l'attenzione del gruppo si concentri sulle zone più favorevoli.

**Il Problema Principale**

- Spesso, a causa della natura dell'algoritmo, tutti i $k$ stati finiscono per convergere e concentrarsi sulla stessa collina locale, perdendo diversità.

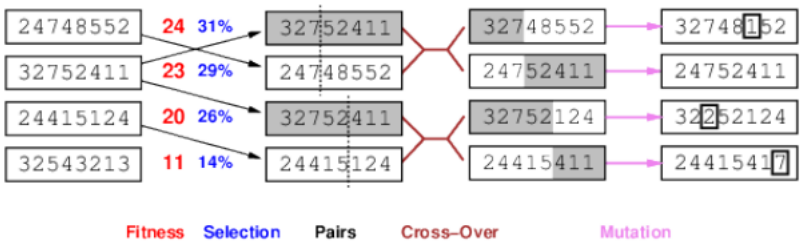
**La Soluzione (Stochastic Beam Search)**

- **Idea:** Invece di scegliere sempre i $k$ successori migliori in modo deterministico, si scelgono $k$ successori in modo **casuale**, dando però una probabilità maggiore (biased) a quelli più validi.
- **Analogia:** Questo comportamento mostra una forte somiglianza con il meccanismo della selezione naturale (simile agli algoritmi genetici).




## 6.5 Genetic algorithms

- **Definizione:** Corrispondono a una ricerca a fascio locale stocastica, in cui i nuovi stati successori non vengono generati tramite singole modifiche, ma combinando coppie di stati esistenti.




**Le Fasi dell'Algoritmo (dal diagramma):**

1. **Fitness:** Valutazione del punteggio di idoneità di ciascuno stato (con assegnazione di percentuali di probabilità di selezione).
2. **Selection:** Scelta degli individui migliori in base alla fitness.

3. **Pairs & Cross-Over:** Abbinamento in coppie e scambio/ricombinazione di parti delle stringhe per generare nuovi discendenti.
4. **Mutation:** Introduzione di piccole modifiche casuali nelle stringhe risultanti.

**Proprietà e Dettagli Chiave:**

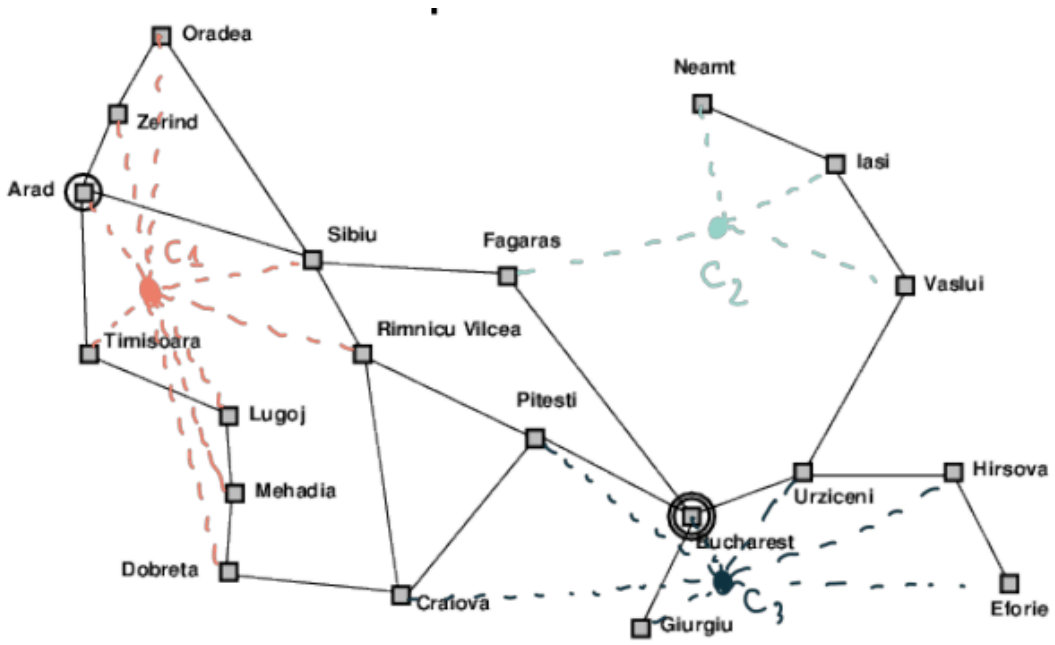
- **Codifica:** Richiedono che gli stati siano rappresentati sotto forma di stringhe (a differenza della Programmazione Genetica - GP - che utilizza veri e propri programmi).
- **Efficacia del Cross-Over:** L'incrocio funziona efficacemente se e solo se le sottostringhe rappresentano componenti autonome e significative del problema (come illustrato nell'esempio delle scacchiere per il problema delle regine).
- **GAs $n$ = Evoluzione:** Gli algoritmi genetici sono solo un'astrazione matematica e non vanno confusi con la vera evoluzione biologica (nella realtà, ad esempio, i geni codificano complessi macchinari di replicazione cellulare).

# 7 Local Search In Continuous Space

Partiamo da un esempio:

### 7.0.1 Example: Placing airports in Romania

**Obiettivo**: localizzare tre aereoporti per minimizzare la somma delle distanze al quadrato per ogni città vicina all'aereoporto.




- **Airport loc.** $(x_1, x_2), (x_2, y_2), (x_3, y_3)$
- **City locations** $(x_c, y_c)$
- $C_a$ cities closest to airport $a \rightarrow (a = \{1, 2, 3\})$
- **objective minimize**: $\sum_a \sum_{c \in C_a} (x_a - x_c)^2 + (y_a - y_c)^2$

#### 7.0.1.1 Come possiamo affrontare questo problema?

**Discretizzare lo spazio**

Rendere lo spazio continuo in uno spazio discreto:

- Scegliere un $\delta$ fisso che rappresenta la dimensione del passo finito
- Considerare $\pm\delta$ spostamenti. Ad esempio, se sei in un grafico a due dimensioni $(x, y)$, i tuoi vicini saranno solo i punti $(x + \delta, y)$, $(x - \delta, y)$, $(x, y + \delta)$ e $(x, y - \delta)$. Invece di avere infiniti vicini, ora sono limitati.
- Ora che lo spazio è diventato discreto possiamo usare qualsiasi algoritmo di ricerca discreto

**Random perturbations**

- Prendiamo il nostro punto corrente e proviamo ad aggiungere piccolo rumore casuale.Continuiamo a generare tentativi casuali finché non ne troviamo uno che migliora lo stato corrente.
  È un approccio «a tentativi» molto simile al first-choice, adattato al continuo generando variazioni a caso finché non becchi la salita.
- **Simulated Annealing (nel continuo)** : il prossimo stato viene scelto in modo casuale (tramite una perturbazione o un campionamento nello spazio continuo), e poi si applica la solita regola di accettazione con la temperatura (es : la formula di Boltzmann) per decidere se accettare anche un peggioramento.

**Gradient (Discesa/Salita del Gradiente)**: Invece di andare a tentativi con i numeri a caso, si calcoli il gradiente della funzione $f(X)$ analiticamente (dove $X = \{x_1, ..., x_n\}$ è il vettore delle coordinate/variabili). Il gradiente ci dà la direzione e il verso matematicamente perfetti in cui la funzione cresce più rapidamente, permettendoci di fare un passo mirato e sicuro.

$$\nabla f(X) = \left( \frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_n} \right)$$

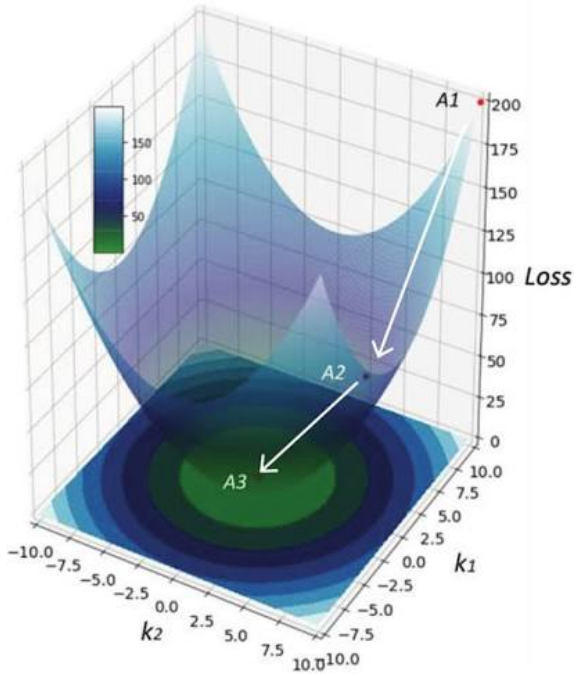
Applicandolo all'esempio dell'aereoporto abbiamo:

- $f(X) = \sum_a \sum_{c \in C_a} (x_a - x_c)^2 + (y_a - y_c)^2$[cite: 6]
  con $X = \{x_1, y_1, x_2, y_2, x_3, y_3\}$ e $n = 6$
- $\frac{\partial f}{\partial x_1} = \sum_{c \in C_1} 2(x_1 - x_c)$
- Per trovare un estremo, si pone l'equazione: $\nabla f(X) = 0$
- Possiamo risolverlo in forma chiusa (**closed form**)?

## 7.1 Local Gradient

A volte non riusciamo a risolvere l'equazione globale $\nabla f(X) = 0$, tuttavia però possiamo analizzare localmente:

- eseguire l'aggiornamento in direzione di salita per ciascuna coordinata
- più ripida è la pendenza (maggiore è la derivata), più ampio è il passo per quella coordinata
- consideriamo una funzione generale $g(x_1, x_2)$ applicando l'aggiornamento per il gradiente:
  - $x_1 \leftarrow x_1 + \alpha * \frac{\partial g(x_1,x_2)}{\partial x_1}, x_2 \leftarrow x_2 + \alpha * \frac{\partial g(x_1,x_2)}{\partial x_2}$
- aggiornamenti in norazione vettoriale:
  - $X = [x_1\, x_2]^T, \quad \nabla g(X) = \left[\frac{\partial g}{\partial x_1}\ \frac{\partial g}{\partial x_2}\right]^T$
  - $X \leftarrow X + \alpha * \nabla g(X)$
- $\alpha$ è la dimensione del passo e trovare un $\alpha$ ottimale è difficile poichè:
  - se troppo grande: può allontanarsi troppo dal minimo ottimale
  - se troppo piccolo: servono più step per trovare un minimo ottimale




## 7.2 Newton-Raphson

- Soluzione efficace per molti problemi di ricerca locale in uno spazio continuo
- Tecnica generale per trovare le radici di una funzione, ovvero per risolvere un'equazione della forma:

$$g(x) = 0$$

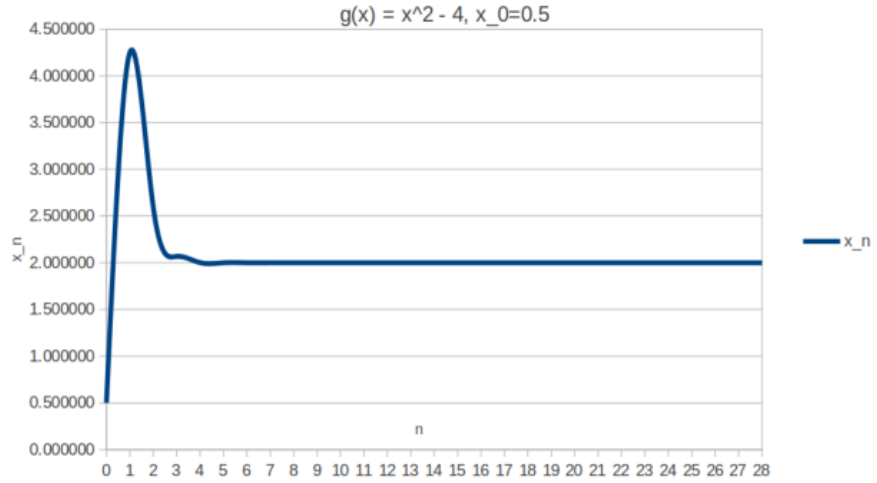
- fissa una stima iniziale della radice $\bar{x}_0$
- aggiorna iterativamente la stima: $\bar{x}_{n+1} = \bar{x}_n - \frac{g(\bar{x}_n)}{g'(\bar{x}_n)}$
- $g'(x) = d\frac{g(x)}{d}x$ è la derivata prima di $g(x)$

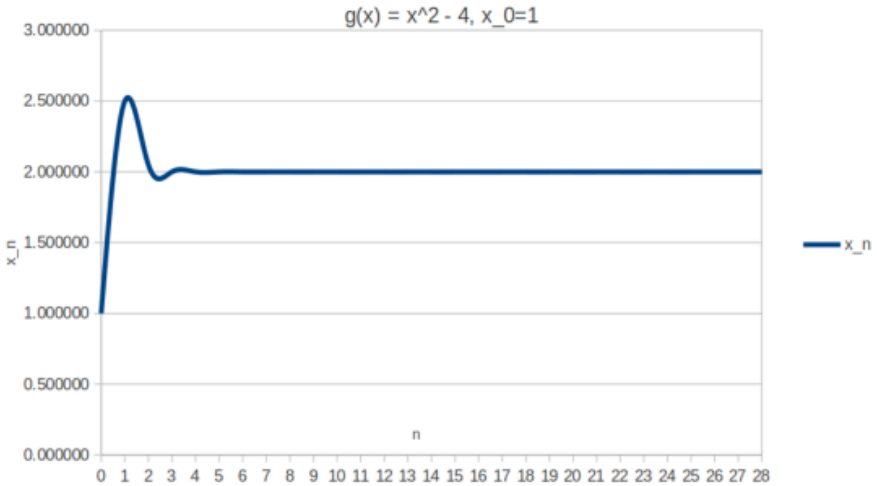
### 7.2.1 Semplice applicazione

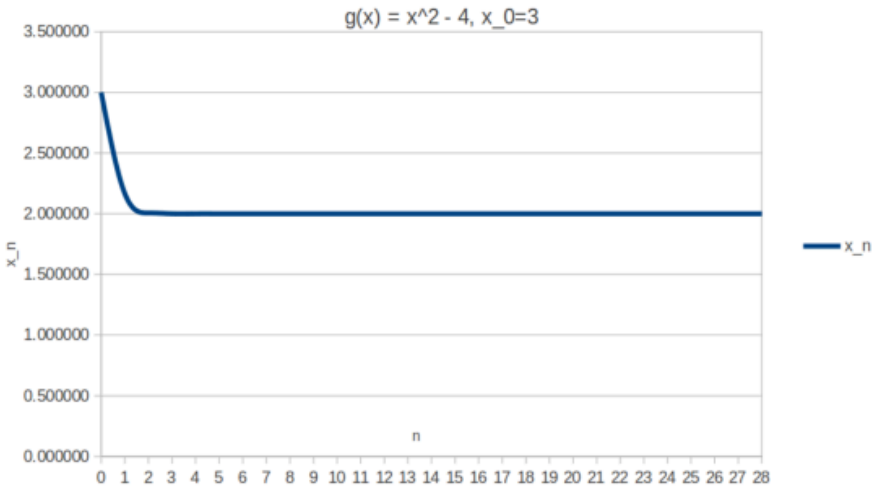
Sia la funzione $g(x) = x^2 - a$
Applicando Newton abbiamo:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

Fissiamo ora a = 4, $x_0 = 1$, risolviamo per $x_i i \in \{1, 2, 3\}$
Come varia $x_i i \in \{1, 2, ...\}$ rispetto a $x_0 \in \{0.5, 1, 3\}$ ?


g(x) = x^2 - 4, x_0=0.5
4.500000
4.000000
3.500000
3.000000
2.500000
2.000000
1.500000
1.000000
0.500000
0.000000
x_n
n
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
x_n


g(x) = x^2 - 4, x_0=1
3.000000
2.500000
2.000000
1.500000
1.000000
0.500000
0.000000
x_n
n
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
x_n


g(x) = x^2 - 4, x_0=3
3.500000
3.000000
2.500000
2.000000
1.500000
1.000000
0.500000
0.000000
x_n
n
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
x_n

## 7.3 Interpretazione geometrica del metodo di Newton

- Supponi che $y = f(x)$ abbia una radice in $x = r$;
- La retta tangente nel punto $(a, f(a))$ è $y = f(a) + (x - a)f'(a)$
- Considera $b$ come l'intercetta della retta tangente, $0 = f(a) + (x - a)f'(a)$ allora:

$$b = a - \frac{f(a)}{f'(a)}$$

- $b$ sarà la stima successiva per il metodo di Newton partendo da $a$
- Nota: cambiamenti rapidi (forti non linearità) di $f(x)$ in $a$ possono causare una stima errata del punto successivo
  - $f(x)$ deve essere liscia (continua e derivabile), di solito assumiamo che $f''(x)$ sia continua vicino a $r$
- La scelta di $x_0$ è cruciale!

## 7.4 Applicazioni con il gradiente

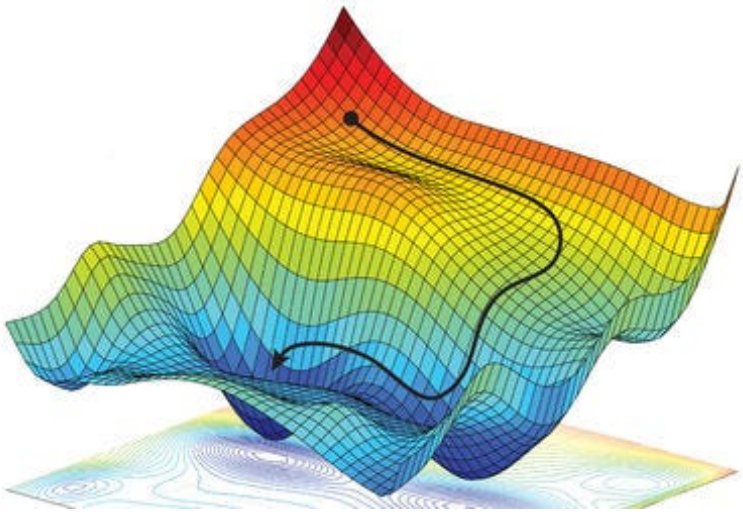
- per trovare un estremo di $f(X)$ vogliamo trovare $\overline{X}$ tale che $\nabla f\left(\overline{X}\right) = 0$
- di conseguenza possiamo usare il metodo di Newton, dove $g(X)$ è $\nabla f(X)$
- in questo caso l'equazione di aggiornamento in forma matrice-vettore diventa:
  - $x \leftarrow x - H_f^{-1}(x)\nabla f(x)$
  - dove $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$
- per problemi ad alta dimensionalità, calcolare le $n^2$ entrate della matrice Hessiana e invertire la matrice $N \times N$ può essere proibitivo. Spesso vengono usate versioni approssimate.

**Ricorda**

- Questo è un metodo **locale**, quindi soffre di massimi locali, altipiani (**plateau**), ecc.
- Il riavvio casuale (**random restart**) e il **simulated annealing** possono essere utili anche negli spazi continui

### 7.4.1 Empirical Gradient

- A volte possiamo calcolare $f(X)$ per un dato input, ma non possiamo calcolare $\nabla f(X) = 0$ nemmeno localmente
  - Ad esempio, la posizione di un insieme di aeroporti richiede una simulazione su larga scala
- **Gradiente Empirico:** risposta di $f(X)$ a un piccolo incremento o decremento di $X$
- **Eseguire una ricerca del gradiente empirico:** equivale a un hill-climbing in una versione discretizzata dello spazio di ricerca



# 8 Constraint Satisfaction Problem

## 8.1 Standard Search Problem

Nei problemi di ricerca standard:

- **State**: è una «scatola nera» (**black-box**), ovvero può essere rappresentato con una qualsiasi struttura dati.
- **Goal test**: può essere una qualsiasi funzsione sugli stati che verifica se l'obiettivo è stato raggiunto.
- **Successor function**: può essere una qualsiasi funzione che associa a uno stato un insieme di stati successori.

## 8.2 Constraint Satisfaction Problems

Nei **CSP** invece lo stato ha una struttura precisa:

- **Variabili**: un insieme $X = \{X_1, ..., X_n\}$ descrive il problema.
- **Domini**: ogni variabile $X_i$ può assumere valori nel proprio dominio $D_i$. L'insieme dei domini è $D = \{D_1, ..., D_n\}$.
- **Goal test**: è un insieme di **vincoli** (**constraints**) che specificano le combinazioni di valori ammesse per sottoinsiemi di variabili.

**Una soluzione è un assegnamento di un valore a ogni variabile che rispetta tutti i vincoli.**

**Key point**

La struttura esplicita di **variabili, domini e vincoli** permette di usare strategie generali di risoluzione per i CSP, sfruttando queste informazioni per essere più efficienti degli approcci di ricerca standard.

### 8.2.1 CSP Example : Map-Coloring




siano:

- Variabili = $WA, NT, Q, NSW, V, SA, T$
- Dominio = $D_i$ = {red, green, blue}
- Vincoli = le regioni adiacenti devono essere di colore diverso, per esempio: $WA \neq NT$ (se il linguaggio lo permette), oppure $(WA, NT) \in$ {(red, green), (red, blue), (green, blue), ...}




Le soluzioni sono assegnazioni che soddisfano tutti i vincoli, ad esempio:

$$\{ \text{Wa} = \text{red}, \text{NT} = \text{green}, \text{Q} = \text{red}, \text{NSW} = \text{green}, \text{V} = \text{red}, \text{SA} = \text{blue}, \text{T} = \text{green} \}$$

### 8.2.2 Example: N-Queens

**Obiettivo**: posizionare $N$ regine su una scacchiera $N \times N$ in modo che nessuna coppia di regine si attacchi, ovvero che non condividano riga, colonna o diagonale.

**Prima formulazione: una variabile per ogni casella**

- **Variabili**: $X_{i,j}$ per ogni $i, j \in \{1, ..., N\}$. Abbiamo quindi $N^2$ variabili.
- **Domini**: $D_{i,j} = \{0, 1\}$, dove 1 indica che nella casella è presente una regina e 0 che è vuota.
- **Vincoli**: per ogni coppia di caselle distinte sulla stessa riga, colonna o diagonale, le combinazioni ammesse sono:

$$(X_{i,j}, X_{r,c}) \in \{(0,0), (0,1), (1,0)\}$$

La coppia $(1, 1)$ è esclusa perché le due regine si attaccherebbero. Il vincolo si applica quando:

- **Stessa riga**: $i = r$
- **Stessa colonna**: $j = c$
- **Stessa diagonale**: $|i - r| = |j - c|$, considerando entrambe le direzioni.

Questi vincoli impongono **al massimo una regina** per riga, colonna e diagonale. Per avere esattamente $N$ regine sulla scacchiera aggiungiamo:

$$\sum_{i=1}^{N} \sum_{j=1}^{N} X_{i,j} = N$$

### 8.2.3 N-Queens: alternative formulation

**Idea**: consideriamo direttamente **una regina per ogni colonna**.

- **Variabili**: $X = \{x_1, ..., x_N\}$, dove $x_i$ rappresenta la **riga** della regina nella colonna $i$. Ora abbiamo solo $N$ variabili.
- **Domini**: $D_i = \{1, ..., N\}$, ovvero tutte le righe della scacchiera.
- **Vincoli**: uno per ogni coppia di variabili, per un totale di $\frac{N(N-1)}{2}$ vincoli. Per ogni $i < j$ deve valere:

$$x_i \neq x_j \quad \wedge \quad |x_i - x_j| \neq |i - j|$$

Il primo impedisce che due regine siano sulla **stessa riga**, il secondo sulla **stessa diagonale**.

**Esempio con $N = 4$**: possiamo anche elencare esplicitamente le coppie di valori ammesse:

$$C_1 : (x_1, x_2) \in \{(1,3), (1,4), (2,4), (3,1), (4,1), (4,2)\}$$

$$C_2 : (x_1, x_3) \in \{(1,2),(1,4),(2,1),(2,3),(2,4),(3,2),(3,4),(4,2),(4,3)\}$$

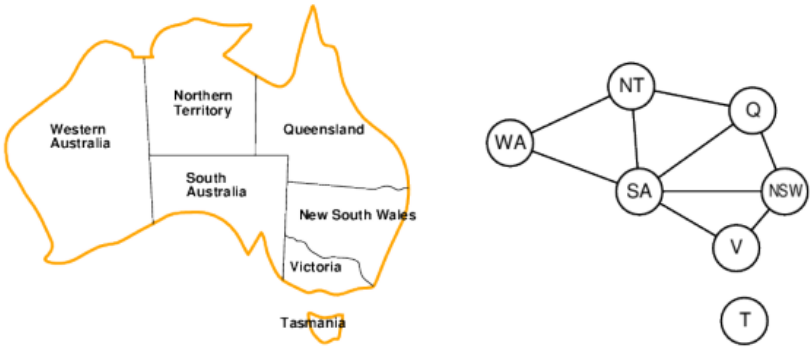
Ad esempio, $(x_1, x_2) = (1,3)$ è ammesso perché le due regine sono su righe diverse e la differenza tra le righe è 2, mentre quella tra le colonne è 1.

**Key point**

Una buona formulazione rende alcuni vincoli **impliciti**: con una variabile per colonna è già garantito che ci sia esattamente una regina in ogni colonna, quindi non serve controllare questo vincolo né aggiungere quello sul numero totale di regine.

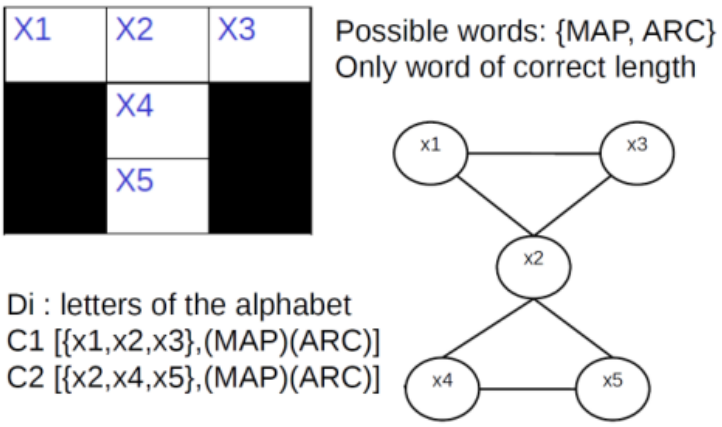
## 8.3 Constraint graph

Grafo primale (primal graph): un nodo per ogni variabile, un arco per ogni vincolo che intercorre tra le due variabili

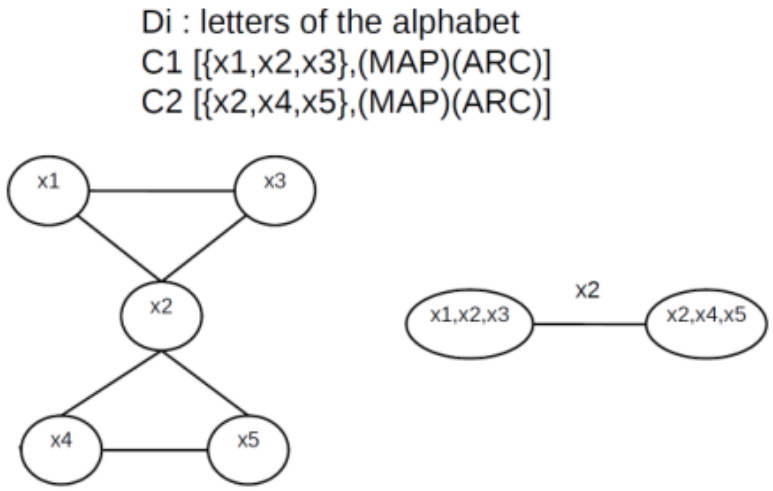



### 8.3.1 Example: Crossword

Data la configurazione di un cruciverba e le possibili parole, assegnare le lettere alle caselle in modo da corrispondere alla configurazione data.



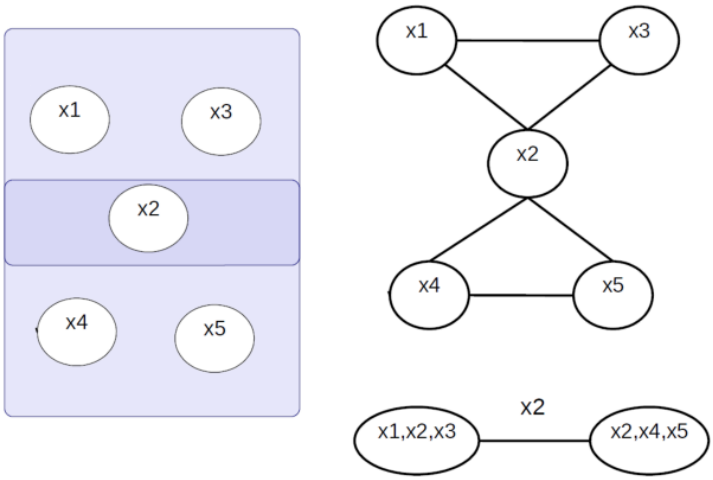
#### 8.3.1.1 Non binary constraints




#### 8.3.1.2 Hypergraphs and Dual Graphs

**Relazione tra hypergraphs e binary graphs**

- Possiamo sempre convertire un hypergraph in un binary graph
- Ogni variabile ha un dominio esponenzialmente più grande




## 8.4 Una prospettiva più ampia del CSP

I **problemi combinatori** richiedono di cercare, in un insieme di possibili soluzioni, una soluzione ammissibile oppure quella migliore secondo un criterio. Lo spazio delle possibilità può essere enorme, rendendo una ricerca esaustiva impraticabile.

### 8.4.1 Graphical Models

Possiamo distinguere diversi tipi di problemi combinatori:

- **Decisione**: esiste un modo di colorare un grafo con $k$ colori rispettando tutti i vincoli? La risposta è **sì/no**.
- **Ottimizzazione**: colorare un grafo con $k$ colori minimizzando il numero di conflitti.
- **Ottimizzazione multiobiettivo**: ottimizzare più criteri, ad esempio scegliere un portafoglio di investimenti minimizzando il rischio e massimizzando il rendimento.

I **Graphical Models** offrono una rappresentazione generale di questi problemi:

- un insieme di **variabili**, **domini** e **funzioni locali** (ad esempio vincoli);
- una **funzione globale** ottenuta aggregando le funzioni locali;

- una soluzione è un **assegnamento delle variabili** che ottimizza la funzione globale.

## 8.5 Mathematical formulation of Constraint Networks

Una **Constraint Network** (CN), ovvero una rete di vincoli, è una tripla:

$$\text{CN} = (X, D, C)$$

- $X = \{x_1, ..., x_n\}$: insieme delle **variabili**.
- $D = \{D_1, ..., D_n\}$: insieme dei **domini**, dove $D_i$ contiene i valori ammessi per $x_i$.
- $C = \{(S_1, R_1), ..., (S_m, R_m)\}$: insieme dei **vincoli**.

Ogni vincolo $(S_i, R_i)$ è composto da:
- **Scope** $S_i \subseteq X$: le variabili coinvolte nel vincolo.
- **Relazione** $R_i$: l'insieme delle tuple di valori ammesse per le variabili dello scope.

Se $S_i = \left\{x_{i_1}, ..., x_{i_k}\right\}$, fissato un ordine delle variabili, allora:

$$R_i \subseteq D_{i_1} \times ... \times D_{i_k}$$

**Esempio**: per due variabili con dominio $\{1, 2\}$ e vincolo $x_1 \neq x_2$, lo scope è $\{x_1, x_2\}$ e la relazione è $\{(1, 2), (2, 1)\}$.

Una **soluzione** assegna un valore a tutte le variabili rispettando tutti i vincoli. Su una rete possiamo svolgere diversi task:
- verificare se **esiste una soluzione** (**consistency check**);
- trovare **una soluzione** oppure **tutte le soluzioni**;
- **contare** le soluzioni;
- trovare la **soluzione migliore**, se è specificato un criterio di ottimizzazione.

### 8.5.1 Solutions and partial solutions

Una **soluzione parziale** è un assegnamento di valori a un sottoinsieme delle variabili.

Una soluzione parziale è **consistente** se soddisfa tutti i vincoli il cui scope contiene **solo variabili già assegnate**. Gli altri vincoli non sono ancora completamente verificabili.

**Ricorda**

Un assegnamento parziale **consistente** non è necessariamente estendibile a una soluzione completa: le variabili rimaste potrebbero non avere alcun valore compatibile.

**Esempio: 4-Queens**
- Variabili $x_1, ..., x_4$: una per colonna, con dominio $\{1, 2, 3, 4\}$.
- Sei vincoli: uno per ogni coppia di regine.
- L'assegnamento $\{x_1 = 1, x_2 = 4, x_3 = 2\}$ è **parziale e consistente**: le tre regine non si attaccano.
- Tuttavia, nella quarta colonna le righe $1, 2, 4$ sono già occupate e la riga 3 è attaccata in diagonale. Quindi questo assegnamento **non può essere completato**.
- Un esempio di soluzione completa è $\{x_1 = 2, x_2 = 4, x_3 = 1, x_4 = 3\}$.

## 8.6 Standard search formulation for CSP

Possiamo formulare un CSP come un problema di ricerca standard:

- **Stati**: gli assegnamenti effettuati fino a quel momento.
- **Stato iniziale**: l'assegnamento vuoto {}.
- **Successor function**: assegnare a una variabile non ancora assegnata un valore che non sia in conflitto con l'assegnamento corrente.
- **Goal test**: l'assegnamento è completo. La consistenza è mantenuta durante la generazione dei successori.

Se non esistono assegnamenti legali per proseguire, quel ramo **fallisce**: bisogna tornare indietro e cambiare una scelta precedente.

**Proprietà di questa formulazione:**

- È la stessa per **tutti i CSP**.
- Con $n$ variabili, ogni soluzione si trova a **profondità $n$**: una scelta naturale è la **Depth-First Search**.
- Il **percorso è irrilevante**: conta l'assegnamento finale. È quindi possibile anche una formulazione a stati completi, come nella ricerca locale.

**Il problema della formulazione ingenua**: a ogni passo possiamo scegliere sia quale variabile assegnare sia quale valore darle. Con domini di dimensione $d$, a profondità $\ell$ il fattore di ramificazione è, nel caso peggiore:

$$b = (n - \ell)d$$

Ignorando le potature dovute ai vincoli, il numero di foglie diventa:

$$(nd)((n-1)d)...(1d) = n!d^n$$

Il fattore $n!$ deriva dai diversi **ordini di assegnamento**: raggiungiamo più volte lo stesso assegnamento finale.

## 8.7 Backtracking search

**Idea**: gli assegnamenti a variabili diverse sono **commutativi**. Assegnare prima $WA = \text{red}$ e poi $NT = \text{green}$ produce lo stesso risultato dell'ordine inverso.

In ogni nodo scegliamo quindi **una sola variabile non assegnata** e generiamo un ramo per ciascun suo valore ammissibile. La scelta della variabile può essere fissa oppure dipendere dall'assegnamento corrente.

- **Fattore di ramificazione**: al massimo $b = d$.
- **Numero di foglie**: al massimo $d^n$, eliminando il fattore $n!$ dovuto agli ordini ridondanti.
- **Backtracking search**: è una ricerca in profondità per CSP che assegna una variabile alla volta e annulla le scelte quando un ramo fallisce.
- È l'algoritmo di base **non informato** per i CSP; nelle slide viene indicato che può risolvere N-Queens per $n \approx 25$.
- L'**ordine di scelta delle variabili** influisce sulle prestazioni, perché può far emergere prima o dopo i conflitti.

### 8.7.1 Pseudo codice

```
function Backtracking-Search(csp) returns a solution or failure
    return Backtrack({}, csp)

function Backtrack(assignment, csp) returns a solution or failure
    if assignment is complete then return assignment
    var <- Select-Unassigned-Variable(assignment, csp)

    for each value in Order-Domain-Values(var, assignment, csp) do
        if value is consistent with assignment then
            add {var = value} to assignment
            inferences <- Inferences(csp, var, value)

            if inferences != failure then
                add inferences to assignment
                result <- Backtrack(assignment, csp)
                if result != failure then return result

            undo {var = value} and all effects of inferences

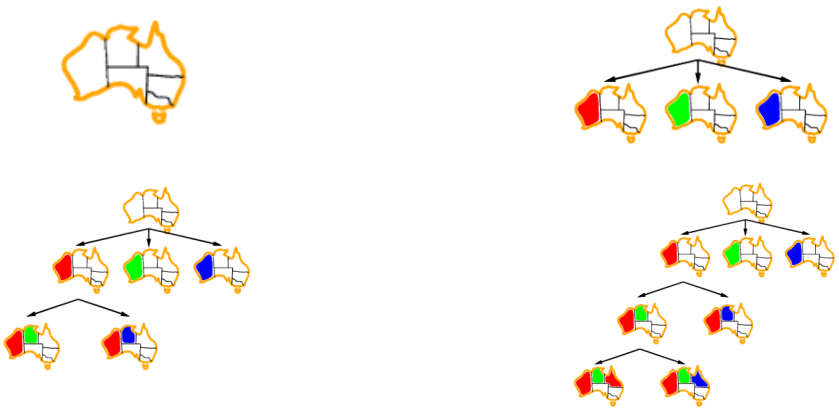
    return failure
```

**Come funziona:**

1. Se l'assegnamento è completo, restituiamo la soluzione.
2. Scegliamo una variabile non assegnata e proviamo i valori del suo dominio in un certo ordine.
3. Se il valore è compatibile con l'assegnamento corrente, lo assegniamo ed eseguiamo le **inferenze**, per dedurre conseguenze della scelta ed eventualmente rilevare un fallimento.
4. Se le inferenze non falliscono, proseguiamo ricorsivamente.
5. Se il tentativo fallisce, **annulliamo l'assegnamento e gli effetti delle inferenze**, comprese eventuali riduzioni dei domini, e proviamo il valore successivo.
6. Se tutti i valori falliscono, restituiamo **failure** al livello precedente.

**Key point**

Il backtracking costruisce un assegnamento una variabile alla volta. Quando una scelta non permette di arrivare a una soluzione, torna indietro e prova un'alternativa. **Scelta delle variabili, ordine dei valori e inferenza** sono i punti in cui possiamo migliorare la ricerca.



Nell'esempio ogni livello dell'albero aggiunge il colore di una regione. Dopo aver assegnato $WA =$ red, per $NT$ possiamo scegliere verde o blu; i rami incompatibili con i colori già assegnati vengono esclusi.

## 8.8 Improving Backtracking

**Obiettivo**: ridurre la dimensione dello spazio di ricerca esplorato, riconoscendo prima le scelte che non possono portare a una soluzione.

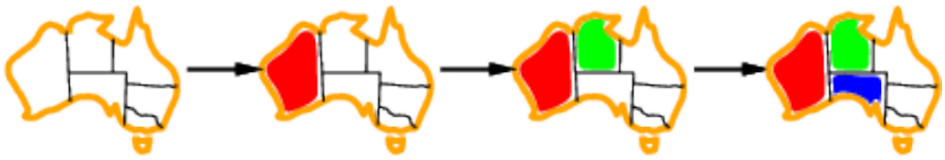
Le principali possibilità sono:

- **Ordinare variabili e valori**: scegliere quale variabile assegnare e in quale ordine provarne i valori.
- **Consistenza locale**: applicare tecniche come **arc consistency** e **path consistency** per eliminare combinazioni incompatibili.
- **Look-ahead**: prevedere come una scelta limita gli assegnamenti futuri e anticipare le inconsistenze.
- **Look-back**: decidere a quale scelta precedente tornare quando si incontra un fallimento.
- **Tree decomposition**: sfruttare la struttura del problema per scomporlo in parti più semplici.

### 8.8.1 Ordering Variables: Minimum Remaining Values

**MRV**: scegliamo la variabile non ancora assegnata con il **minor numero di valori legali rimasti**, tenendo conto dell'assegnamento corrente.

- Se una variabile ha un solo valore disponibile, affrontiamo subito quella scelta obbligata.
- Se non ha più valori disponibili, scopriamo immediatamente che il ramo fallisce.
- L'idea è assegnare prima le variabili **più difficili**, per evitare di scoprire un conflitto solo dopo molte altre assegnazioni.

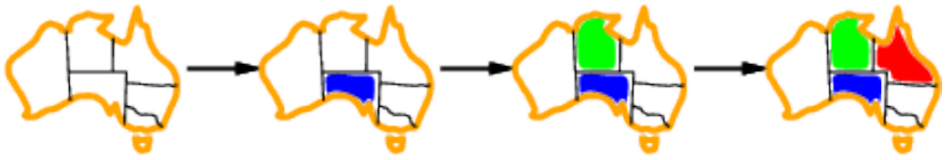


**Esempio**: dopo $WA = \text{red}$ e $NT = \text{green}$, $SA$ può essere soltanto blu, perché confina con entrambe. MRV sceglie quindi $SA$ prima delle regioni che hanno ancora più colori disponibili.

### 8.8.2 Ordering Variables: Degree heuristic

In caso di **parità tra variabili secondo MRV**, scegliamo quella coinvolta nel **maggior numero di vincoli con le variabili ancora non assegnate**.

L'idea è affrontare prima una variabile che limita molte altre scelte, così da vincolare prima il resto del problema.



**Esempio**: all'inizio tutte le regioni hanno tre colori disponibili. Scegliamo $SA$, perché confina con cinque regioni ed è quindi quella con più vincoli sulle variabili rimanenti.

**MRV e Degree heuristic sono generali**: possono essere applicate a qualsiasi CSP, sfruttando domini e vincoli.

### 8.8.3 Value ordering: Least Constraining Value

Dopo aver scelto la variabile, dobbiamo decidere **quale valore provare per primo**.

La **Least Constraining Value** (LCV) preferisce il valore che **esclude meno valori dai domini delle variabili non ancora assegnate**, lasciando più possibilità per completare la soluzione.




**Esempio**: siano $WA =$ red e $NT =$ green. Per $Q$ possiamo scegliere rosso o blu:

- $Q =$ red: lascia il blu disponibile per $SA$.
- $Q =$ blue: $SA$ confina con regioni che usano tutti e tre i colori, quindi non ha più valori disponibili.

LCV preferisce quindi **rosso** per $Q$.

**Ricorda**

- **MRV**: scegli la variabile con meno valori legali rimasti.
- **Degree heuristic**: a parità di MRV, scegli quella con più vincoli sulle variabili rimanenti.
- **LCV**: per la variabile scelta, prova prima il valore che lascia più libertà alle altre.

Combinando queste euristiche, nelle slide viene indicato che il problema delle **1000 regine** diventa affrontabile. Il miglioramento effettivo dipende però dal problema specifico.

## 8.9 Inference: make hidden constraints explicit

Possiamo ragionare sui vincoli esistenti per **dedurne altri**, rendendo esplicite informazioni già implicite nel problema.

**Esempio: colorazione di un grafo con due colori**

- Variabili: $x_1, x_2, x_3$.
- Domini: $D_1 = D_2 = D_3 = \{R, B\}$.
- Vincoli iniziali: $C_1 : x_1 \neq x_2$ e $C_2 : x_2 \neq x_3$.

Se $x_2 = R$, allora $x_1 = x_3 = B$; se $x_2 = B$, allora $x_1 = x_3 = R$. Possiamo quindi dedurre:

$$C_3 : x_1 = x_3$$

Non serve esplorare assegnamenti in cui $x_1 \neq x_3$, perché non potranno soddisfare entrambi i vincoli iniziali. Questa deduzione dipende dall'avere **solo due colori**.

### 8.9.1 Trade-off between inference and search

- **Vincoli più restrittivi**: riducono lo spazio di ricerca. Aggiungere vincoli dedotti correttamente preserva tutte le soluzioni ed elimina prima i tentativi inutili.
- **Costo dell'inferenza**: dedurre e mantenere i nuovi vincoli richiede calcolo.
- **Costo dei controlli**: ogni nuova assegnazione può richiedere la verifica di più vincoli.

Considerando al massimo una relazione aggregata per ciascuno scope:

- con **vincoli binari**, una nuova variabile può richiedere fino a $O(n)$ controlli con quelle già assegnate;
- con **vincoli di arità $r$**, gli scope che coinvolgono la nuova variabile possono essere $O(n^{r-1})$.

Questi sono conteggi dei **vincoli da controllare**: il costo di ciascun controllo dipende anche da come è rappresentata la relazione.

### 8.9.2 Backtrack Free Search

Una rete è **backtrack free**, rispetto all'ordine di assegnamento considerato, se ogni assegnamento parziale consistente generato può essere esteso a una soluzione: non ci sono rami consistenti che terminano in un fallimento.

Una DFS può quindi raggiungere un assegnamento completo e consistente **senza dover tornare indietro**.

**Esempio**: nella rete con due colori, consideriamo l'ordine $x_1, x_3, x_2$.

- Senza $C_3$, la scelta $x_1 = R, x_3 = B$ sembra consistente, ma non lascia alcun colore per $x_2$.
- Aggiungendo $C_3 : x_1 = x_3$, quella scelta viene esclusa subito. Se $x_1 = x_3 = R$, basta assegnare $x_2 = B$, e viceversa.

In questo esempio il vincolo dedotto rende la ricerca backtrack free. **In generale, aggiungere qualche vincolo non basta a garantire questa proprietà.**

## 8.10 Look-Ahead

Il **look-ahead** anticipa l'effetto di una scelta sulle variabili future, usando inferenze locali come la consistenza d'arco o di cammino.

- Prevede **come assegnare un valore restringe le possibilità successive**.
- Aggiorna i domini delle variabili ancora non assegnate.
- Fornisce informazioni utili per scegliere la prossima variabile e il prossimo valore, ad esempio il numero di valori legali rimasti per MRV.

Le due strategie considerate sono:

- **Forward Checking**: controlla separatamente le variabili future rispetto all'assegnamento appena effettuato.

- **Arc consistency look-ahead**: propaga la consistenza d'arco, considerando anche le compatibilità tra variabili non ancora assegnate.

### 8.10.1 Costi e benefici

Il look-ahead comporta un **costo aggiuntivo a ogni assegnazione**, perché bisogna propagare i vincoli. Può però ridurre molto lo spazio esplorato:

- scopre prima che una scelta rende impossibile completare un sottoproblema;
- elimina valori incompatibili dai domini futuri;
- evita di scendere in rami destinati a fallire.

In generale non elimina il comportamento esponenziale nel **caso peggiore**: bisogna bilanciare il costo della propagazione con il risparmio nella ricerca.

### 8.10.2 Forward Checking

È una forma limitata di **propagazione dei vincoli**: dopo aver assegnato una variabile, ne propaghiamo l'effetto su ciascuna variabile futura collegata da un vincolo, **separatamente**.

**Come funziona nei CSP binari:**

1. Assegniamo $x_i = v$.
2. Per ogni variabile non assegnata $x_j$ vincolata a $x_i$, eliminiamo da $D_j$ i valori incompatibili con $x_i = v$.
3. Se un dominio diventa **vuoto**, il tentativo fallisce: annulliamo la scelta e le riduzioni dei domini, poi proviamo il prossimo valore di $x_i$.
4. Se tutti i domini restano non vuoti, continuiamo la ricerca.

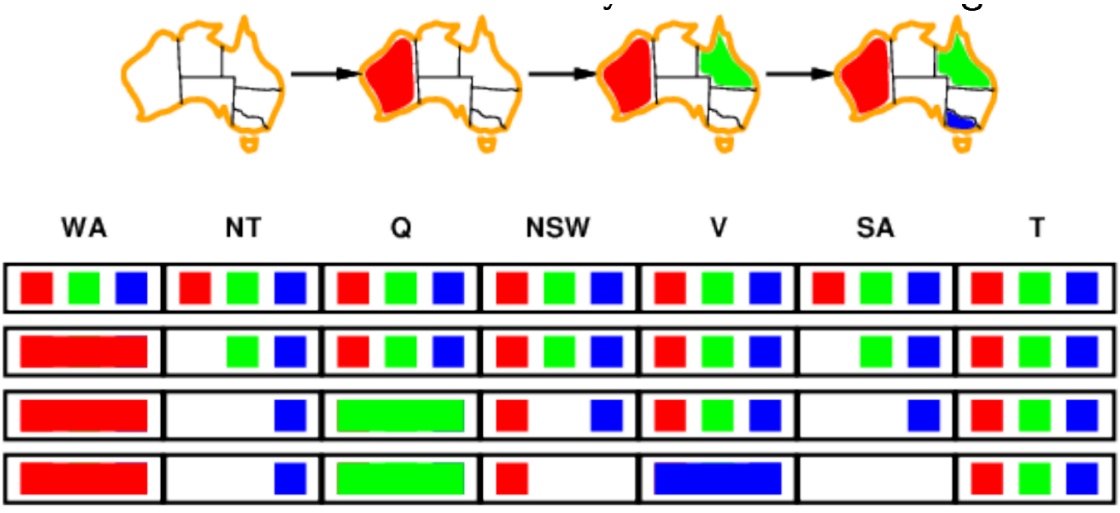
**Esempio**: nel map-coloring, assegnare $SA = \text{blue}$ elimina il blu dai domini di $WA, NT, Q, NSW, V$. Se una di queste regioni aveva soltanto il blu disponibile, il fallimento viene rilevato subito.

**Limite del Forward Checking**

Domini futuri tutti non vuoti **non garantiscono una soluzione**. Il forward checking verifica la compatibilità con la variabile appena assegnata, ma non propaga le incompatibilità tra le variabili ancora non assegnate.

### 8.10.3 Forward Checking: esempio sulla mappa

Manteniamo aggiornati i **valori legali rimasti** per ogni regione. Inizialmente tutti i domini contengono rosso, verde e blu.




1. Assegniamo $WA = \text{red}$: eliminiamo il rosso dai domini di $NT$ e $SA$.

2. Assegniamo $Q = \text{green}$: eliminiamo il verde da $NT, SA, NSW$. Ora $NT$ e $SA$ possono essere soltanto blu, mentre $NSW$ può essere rosso o blu.
3. Proviamo $V = \text{blue}$: eliminiamo il blu da $SA$ e $NSW$. Il dominio di $SA$ diventa **vuoto**, quindi il tentativo fallisce subito.

Non bisogna interrompere tutta la ricerca: **annulliamo il tentativo corrente** e proviamo un altro valore. Già dopo il secondo passo, però, $NT$ e $SA$ sono entrambi obbligati al blu pur essendo adiacenti: il forward checking non rileva questo conflitto tra due variabili non ancora assegnate.

### 8.10.4 Forward Checking: Graph Colouring Example

Consideriamo la rete $R_{GC}$ delle slide, con variabili $x_1, ..., x_7$ e domini:

- $D_1 = \{R, B, G\}$
- $D_2 = D_5 = \{B, G\}$
- $D_3 = D_4 = D_7 = \{R, B\}$
- $D_6 = \{R, G, Y\}$

I colori sono **rosso** $(R)$, **blu** $(B)$, **verde** $(G)$ e **giallo** $(Y)$. Tutti i vincoli impongono colori diversi sulle seguenti coppie:

$$(x_1, x_2), (x_1, x_3), (x_1, x_4), (x_1, x_7), (x_2, x_6),$$

$$(x_3, x_7), (x_4, x_5), (x_4, x_7), (x_5, x_6), (x_5, x_7)$$

**Esecuzione del forward checking:**

1. $x_1 = R$: rimuoviamo il rosso da $D_3, D_4, D_7$, che diventano tutti $\{B\}$.
2. $x_2 = B$: non cambia i domini futuri, perché $D_6$ non contiene il blu.
3. $x_3 = B$: essendo $x_3 \neq x_7$, eliminiamo il blu da $D_7$, che diventa vuoto. Il ramo fallisce.

Il conflitto era già implicito dopo $x_1 = R$, ma il forward checking lo scopre soltanto quando assegniamo $x_3$.

### 8.10.5 Complexity of Forward Checking

Siano $k$ la dimensione massima dei domini ed $e$ il numero di vincoli binari considerati, assumendo costo costante per controllare una coppia di valori.

- Per **un valore fissato** della variabile corrente, controlliamo fino a $k$ valori per ogni variabile futura coinvolta. Sommando i controlli sui vincoli otteniamo un limite di $O(ek)$.
- Per provare fino a $\boldsymbol{k}$ **valori della variabile corrente**, il limite diventa:

$$O(ek^2)$$

È il costo della procedura locale di selezione dei valori con forward checking, **non dell'intera ricerca**, che può ancora esplorare un numero esponenziale di nodi.

## 8.11 Arc Consistency Look-Ahead

La **consistenza d'arco** (**Arc Consistency**, AC) considera anche i vincoli **tra variabili non ancora assegnate**, propagando ulteriormente le riduzioni dei domini.

Un arco diretto $x_i \to x_j$ è **consistente** se ogni valore di $x_i$ ha almeno un valore compatibile nel dominio di $x_j$, detto **supporto**:

$$\forall a \in D_i, \exists b \in D_j : (a, b) \in R_{ij}$$

Se un valore $a \in D_i$ non ha alcun supporto in $D_j$, possiamo **eliminarlo da $D_i$**: non potrà comparire in una soluzione.

**Come funziona il look-ahead con AC:**

1. Rendiamo la rete consistente per archi.
2. Proviamo $x_i = a$, restringendo il suo dominio a $\{a\}$.
3. Propaghiamo le conseguenze sui domini delle altre variabili, fino a quando non ci sono più riduzioni oppure un dominio diventa vuoto.
4. Se un dominio si svuota, annulliamo il tentativo; altrimenti continuiamo la ricerca con i domini aggiornati.

**Forward Checking e Arc Consistency**

- **Forward checking**: elimina i valori incompatibili con la nuova assegnazione dalle variabili future direttamente collegate.
- **AC look-ahead**: propaga anche le conseguenze delle riduzioni tra le variabili future, cercando valori privi di supporto.

### 8.11.1 The power of AC: Sudoku

Possiamo rappresentare un Sudoku come una rete di vincoli:

- **Variabili**: le 81 celle, da A1 a I9.
- **Domini**: $\{1, ..., 9\}$ per le celle vuote; un solo valore per quelle già compilate.
- **Vincoli**: tutti i valori devono essere diversi (**AllDiff**) in ogni riga, colonna e riquadro $3 \times 3$.

Ad esempio:

- Riga A: AllDiff(A1, A2, ..., A9).
- Colonna 1: AllDiff(A1, B1, ..., I1).
- Primo riquadro: AllDiff(A1, A2, A3, B1, B2, B3, C1, C2, C3).

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A | | | 3 | | 2 | | 6 | | |
| B | 9 | | | 3 | | 5 | | | 1 |
| C | | | 1 | 8 | | 6 | 4 | | |
| D | | | 8 | 1 | | 2 | 9 | | |
| E | 7 | | | | | | | | 8 |
| F | | | 6 | 7 | | 8 | 2 | | |
| G | | | 2 | 6 | | 9 | 5 | | |
| H | 8 | | | 2 | | 3 | | | 9 |
| I | | | 5 | | 1 | | 3 | | |

**Esempio sulla colonna 6**: i numeri già presenti sono $5, 6, 2, 8, 9, 3$, quindi alle tre celle vuote $\text{A6}, \text{E6}, \text{I6}$ restano inizialmente i candidati $\{1, 4, 7\}$. I vincoli sulle rispettive righe e riquadri restringono ulteriormente questi domini e le riduzioni possono propagarsi ad altre celle.

Le slide indicano che **questo specifico Sudoku** si risolve con la sola propagazione, senza ricerca. Non è una garanzia per qualsiasi Sudoku. Inoltre, propagare un vincolo globale **AllDiff** può essere più forte che applicare AC alle sole disuguaglianze binarie tra celle.

### 8.11.2 Arc Consistency Look-Ahead Complexity

**AC-1, AC-3 e AC-4** sono algoritmi sistematici per rendere una rete consistente per archi.

Con $e$ vincoli binari e domini di dimensione massima $k$:

- una chiamata ad **AC-4** ha complessità temporale $O(ek^2)$;
- nel look-ahead, se eseguiamo AC-4 separatamente per ciascuno dei $k$ possibili valori della variabile corrente, otteniamo:

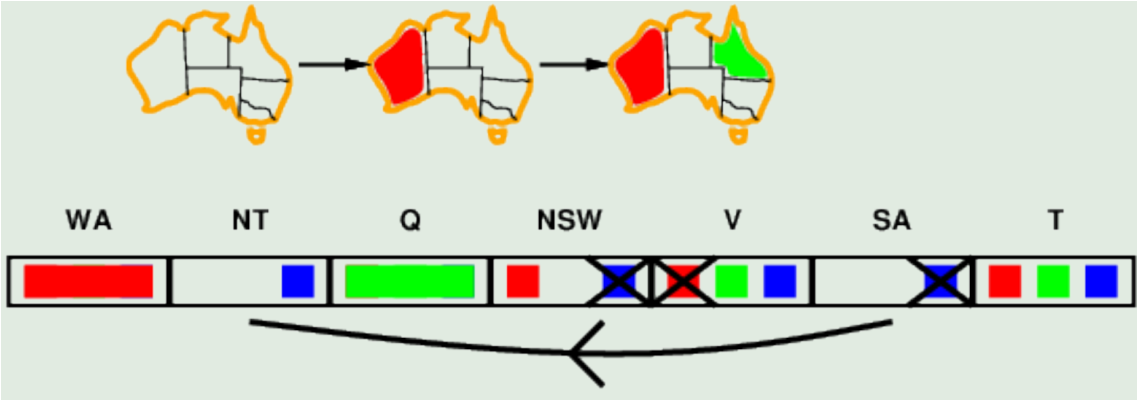
$$O(ek^3)$$

Anche questo è un costo di **propagazione locale**: non comprende tutti i nodi che il backtracking potrebbe visitare.

### 8.11.3 Example of AC Look-Ahead

Riprendiamo l'esempio della mappa, dopo $WA = \text{red}$ e $Q = \text{green}$. Il forward checking lascia:

- $D_{NT} = D_{SA} = \{\text{blue}\}$
- $D_{NSW} = \{\text{red}, \text{blue}\}$
- $D_V = \{\text{red}, \text{green}, \text{blue}\}$




AC usa anche i vincoli tra queste variabili: per esempio, poiché $SA$ può essere soltanto blu, elimina il blu da $NSW$; quindi $NSW$ resta rosso e può a sua volta restringere $V$.

Soprattutto, $NT$ e $SA$ sono adiacenti e hanno entrambi il solo blu: il loro vincolo di disuguaglianza non ha supporti. **La propagazione svuota un dominio e rileva subito il fallimento**, senza dover prima assegnare $V$.

## 8.12 Enforcing Arc Consistency for Networks

Un **vincolo binario** tra $x_i$ e $x_j$ è consistente per archi se lo è in **entrambe le direzioni**: ogni valore di $D_i$ ha un supporto in $D_j$ e viceversa.

Una **rete** è consistente per archi se tutti i suoi vincoli binari lo sono.

La procedura **Revise** controlla un singolo arco $x_i \to x_j$ ed elimina da $D_i$ i valori che non hanno supporto in $D_j$.

**Perché non basta controllare ogni arco una sola volta?** Se riduciamo $D_i$, una variabile vicina potrebbe perdere alcuni supporti che prima esistevano. Dobbiamo quindi ricontrollare gli archi interessati, ripetendo la propagazione fino a un **punto fisso**, cioè finché nessun dominio cambia più.

### 8.12.1 Insights on Arc Consistency

- **Un dominio vuoto** dimostra che il problema corrente è inconsistente.
- **AC e domini tutti non vuoti** non garantiscono, in generale, che esista una soluzione.

**Controesempio**: tre variabili con dominio $\{R, B\}$, collegate a triangolo da vincoli di disuguaglianza. Ogni colore ha un supporto su ciascun singolo arco, quindi la rete è AC; tuttavia, un triangolo non può essere colorato con due colori.

AC controlla la compatibilità locale dei singoli vincoli binari: i supporti trovati su archi diversi potrebbero non essere utilizzabili **tutti insieme**.

**Caso speciale: grafi senza cicli**

Se i vincoli sono **binari**, il grafo primale è **aciclico** e la rete è AC con domini non vuoti, allora esiste una soluzione. Scegliendo una radice e assegnando ogni genitore prima dei figli, possiamo costruirla **senza backtracking**.

L'ordine conta: in un albero, una volta scelto un valore del genitore, AC garantisce un valore compatibile per ciascun figlio; i sottoalberi non hanno vincoli che li colleghino tra loro.

## 8.13 Exploiting problem structure: Cycle Cutset

Un **cycle cutset** di un grafo non orientato è un insieme di nodi la cui rimozione rende il grafo **aciclico**, cioè un albero o una foresta.

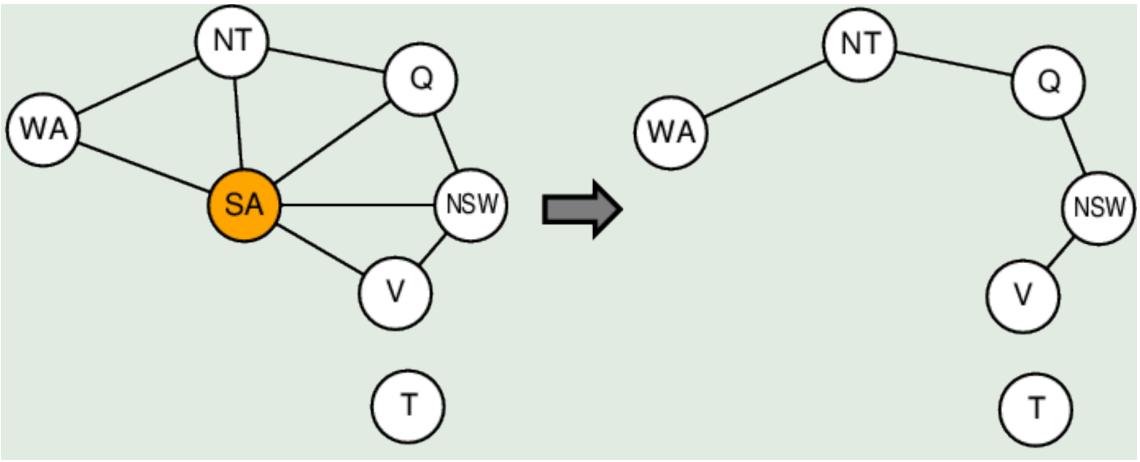
**Conditioning**: dopo aver assegnato una variabile, possiamo rimuoverla dal grafo residuo, mantenendo nei domini dei vicini le restrizioni imposte dal valore scelto.

**Cutset conditioning:**

1. Scegliamo un cycle cutset.
2. Proviamo un assegnamento dei suoi valori, verificando i vincoli interni al cutset.
3. Propaghiamo le restrizioni sui domini delle variabili rimanenti.
4. Il grafo residuo è aciclico: applichiamo AC e, se tutti i domini restano non vuoti, costruiamo una soluzione nell'ordine genitore-figli.
5. Se il tentativo fallisce, ripristiniamo i domini e proviamo un altro assegnamento del cutset.

Con $c$ variabili nel cutset e domini di dimensione massima $k$, ci sono fino a $k^c$ assegnamenti da provare. La parte esponenziale dipende quindi dalla **dimensione del cutset**, mentre ciascun problema residuo su una foresta binaria si risolve in tempo polinomiale.

### 8.13.1 Example: Australia map colouring




Per la mappa australiana basta il cutset $\{SA\}$. Rimuovendo $SA$ rimangono la catena $WA - NT - Q - NSW - V$ e la variabile isolata $T$.

Proviamo un colore per $SA$, lo eliminiamo dai domini delle regioni confinanti e risolviamo la foresta residua. Se serve, ripetiamo con gli altri colori di $SA$.

## 8.14 Esercizi finali

### 8.14.1 AC Look-Ahead sul Graph Colouring Example

Usiamo la rete $R_{GC}$ già descritta e mostriamo perché AC rileva il fallimento **subito dopo $x_1 = R$**:

1. I vincoli con $x_1$ riducono $D_3, D_4, D_7$ a $\{B\}$.
2. Consideriamo $x_3 \neq x_7$: il solo valore $B$ di $x_3$ non ha alcun supporto in $D_7 = \{B\}$.
3. Revise elimina $B$ da $D_3$, che diventa **vuoto**. Il tentativo fallisce immediatamente.

Il forward checking, invece, attendeva l'assegnamento di $x_3$ per scoprire lo stesso conflitto.

### 8.14.2 Cycle Cutset sul Graph Colouring Example

Scegliamo il cutset $\{x_1, x_7\}$. Rimuovendo queste variabili, il grafo residuo contiene la catena $x_2 - x_6 - x_5 - x_4$ e il nodo isolato $x_3$: è quindi una foresta.

**Assegnamenti del cutset:**

- Se $x_1 = R$, i vincoli obbligano $x_3 = x_7 = B$, in conflitto con $x_3 \neq x_7$.
- Se $x_1 = B$, obbligano $x_3 = x_7 = R$, ancora in conflitto.
- Rimane quindi $\boldsymbol{x_1 = G}$, con $x_7 = R$ oppure $x_7 = B$.

Propagando i vincoli nel grafo residuo otteniamo:

- $x_2 = B$, perché $D_2 = \{B, G\}$ e $x_2 \neq x_1$.
- $x_3$ e $x_4$ devono avere il colore opposto a $x_7$, dato che i loro domini sono $\{R, B\}$.
- $x_5 = G$: è diverso sia da $x_4$ sia da $x_7$, che insieme usano rosso e blu.
- $x_6 \in \{R, Y\}$: deve essere diverso da $x_5 = G$; il vincolo con $x_2 = B$ non elimina altri valori dal suo dominio.

Le **quattro soluzioni**, nell'ordine $(x_1, x_2, x_3, x_4, x_5, x_6, x_7)$, sono:

$$(G, B, R, R, G, R, B), \quad (G, B, R, R, G, Y, B)$$

$$(G, B, B, B, G, R, R), \quad (G, B, B, B, G, Y, R)$$

**Key point**

La propagazione sfrutta i **vincoli** per eliminare scelte impossibili; il cutset conditioning sfrutta la **struttura del grafo** per concentrare la ricerca sulle variabili che interrompono i cicli.